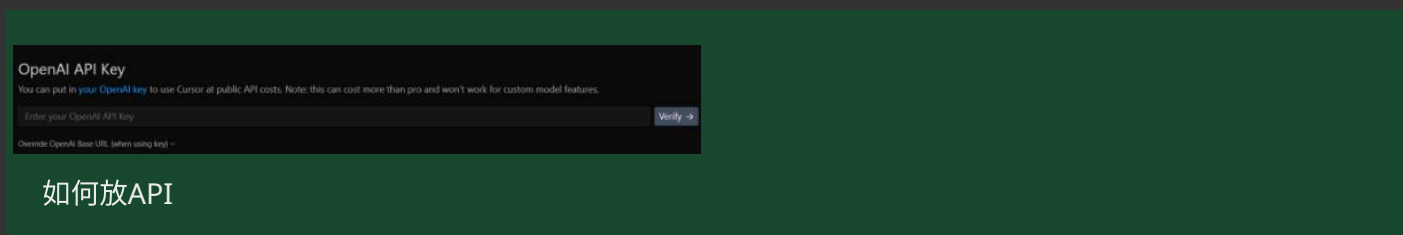


物聯網作業實作問答



如何放API

你這個畫面是 Cursor 裡要你填 **OpenAI API Key** 的地方。做法是：

一、先到 OpenAI 產生 API Key

1. 開瀏覽器到：<https://platform.openai.com>[↗] [OpenAI 平台](#)
2. 用你的 OpenAI 帳號登入（跟 ChatGPT 一樣的帳號也可以）。
3. 登入後，進到「API keys」或「View API keys」頁面。[OpenAI 平台](#)
4. 按 **Create new secret key**（建立新的密鑰）。
5. 給它取個名字（例如：**cursor-key**），按建立。
6. 會跳出一串像 **sk-xxxx...** 很長的字串，**只會顯示一次**，立刻按複製。

⚠ 注意：

- 這串 key 就是「錢包」，不要給別人看，也不要貼到 GitHub、影片、截圖裡。[OpenAI 平台](#)
- 使用 API 是另外計費，不是用你的 ChatGPT Plus 的額度。

二、貼到 Cursor 裡

1. 回到你截圖的 Cursor 畫面：
 - **Settings** → **OpenAI API Key** 那個輸入框。
2. 把剛剛複製的 **sk-** 開頭那串 **貼上** 到 **Enter your OpenAI API Key**。
3. 按右邊的 **Verify** 按鈕。
4. 如果沒有錯字、沒有多空格，Cursor 會顯示驗證成功，之後在 Cursor 裡用 AI 助手時，就會走你自己的 OpenAI API。

三、幾個常見問題

- 貼了顯示錯誤？
 - 確認 key 是完整的（前後沒有少字、沒有空格）。
 - 確認不是過期或被你在 Dashboard 裡刪掉的 key。[OpenAI 平台](#)

- 不想用自己 API，要用 **Cursor Pro** ？
那就把這格留空，只用 Cursor 自帶的額度就好。

如果你願意，可以貼給我（文字就好，不要真的貼出 key 😊）你看到的錯誤訊息內容，我可以幫你判斷是哪裡出問題。

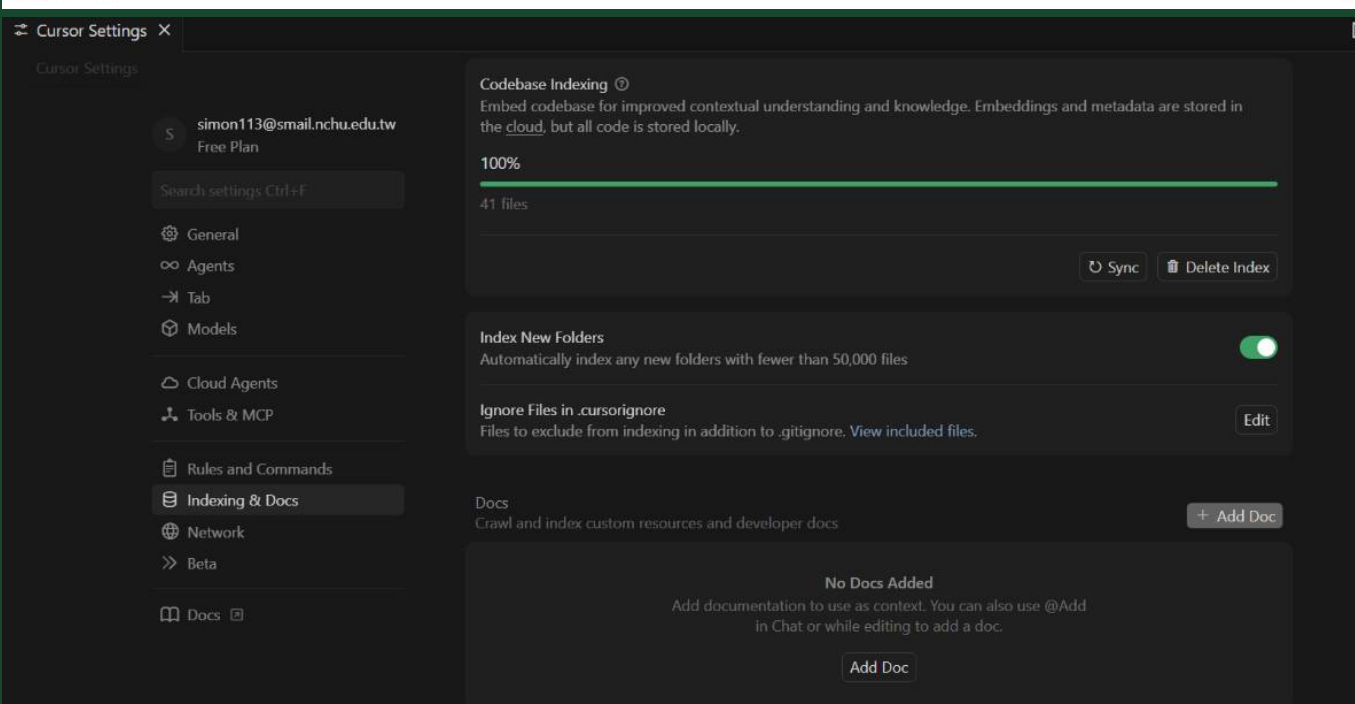
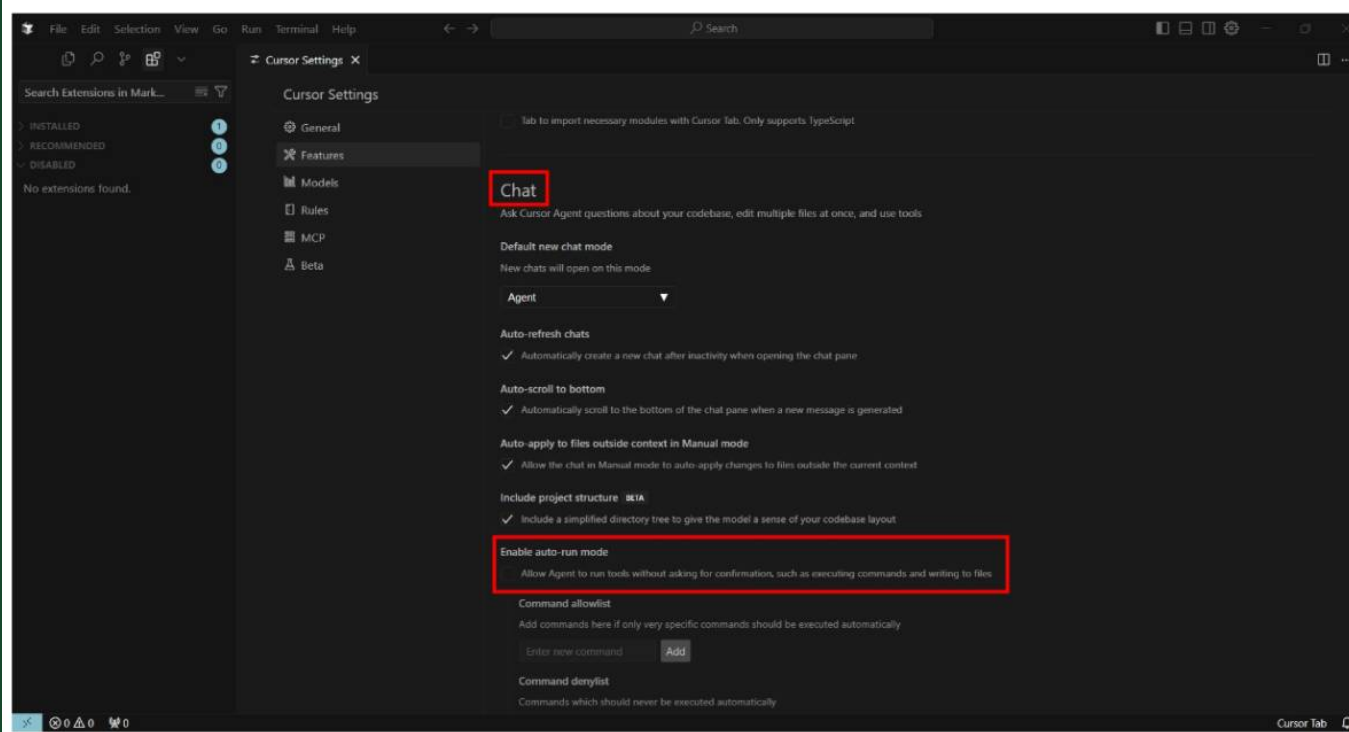


2. 啟用 YOLO (auto-run) 模式

Cursor 有一個很特別的「YOLO 模式」，這個功能可以讓內建的 AI 變得更主動、更積極幫你寫程式。

平常如果你需要請 AI 補齊一段程式碼，還要手動下指令；但開啟 YOLO 模式後，只要簡單描述想做什麼，AI 就會大膽地直接幫你生出完整程式碼，甚至自動運行，幫你把步驟縮短到最少。

點右上角的齒輪圖示進入設定，找到「Features」這個區塊，裡面就有一個「Enable auto-run mode」開關，打開它，設定就完成了。



但我的Cursor找不到

你那個教學截圖是「舊版 Cursor 介面」，所以才會看到
Cursor Settings > Features > Chat > Enable auto-run mode (YOLO)。
 你現在這版 UI 已經改版了，「Features」這個選項不見是正常的。
 用你現在這版要開 YOLO / Auto-run 可以這樣找：

1. 打開 Cursor Settings

- 右上角齒輪 → **Cursor Settings**
- 或快捷鍵：**Ctrl + Shift + J**。

2. 在左邊改點「Agents」

你截圖現在是在 **Indexing & Docs**，請往上點那個 **Agents**。

3. 在 Agents 裡找到 Auto-Run 區塊

進去之後往下捲，會看到類似這幾個設定（名稱可能略有差）：

- **Enable auto-run mode**（或 Auto-Run）→ 把開關打開
- **Command allowlist**
- **Command denylist** [Cursor+1](#)

4. （可選）設定哪些指令可以自動跑

- 在 **allowlist** 裡加你常用又安全的指令開頭，例如：
 - **npm、yarn、python、pytest、npm run dev...**
- 危險指令（例如 **rm, git reset --hard**）不要放進去，
 甚至可以放到 **denylist**，這樣就算 YOLO 也一定會再問你一次。

5. 確認 Chat 是 Agent 模式

Auto-run 只對 **Agent Chat** 有用：

- 打開右側 Chat 面板，點右上角「...」或齒輪 → **Chat Settings**，
- 把 **Default new chat mode** 設成 **Agent**。 [CSDN+1](#)

如果你進到 **Agents** 還是完全看不到 Auto-Run / YOLO：

- 先到 Cursor 官網重新下載最新版安裝檔覆蓋更新。 [Cursor](#)
- 更新完再照上面步驟到 **Agents** 裡找一次。

你可以先照這流程點看看，卡在某一步（例如 Agents 裡長得不一樣、完全沒有 Auto-Run 那一段），截一張那個畫面給我，我再對著你的畫面一步一步教你開。



```
st.markdown("#### Lucky Vicky API 設定")
provider_options = {
    "groq": {
```

```

    "env": "GROQ_API_KEY",
    "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
},
"openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
"mistral": {"env": "MISTRAL_API_KEY", "models": ["ministral-8b-latest", "mistral-large-latest"]},
}
provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])
api_key = st.text_input(f"{provider_options[provider]['env']} (不會儲存)", type="password")
if api_key:
    os.environ[provider_options[provider]["env"]] = api_key.strip()

```

我要改寫 `api_key` 我的金鑰 `h97MT06BhbTrKUqDzEq4zjVenhpc32Bm` 請問我要寫到哪裡

先講最重要的一件事：

你剛剛把「完整金鑰」貼出來了，這代表它**已經視同外洩**，建議馬上去那個服務的後台把這組 key 刪掉／重新產生一組新的，再用新的那組來測試，舊的就當報廢。

回到你的程式碼：

```

python

api_key = st.text_input(f"{provider_options[provider]['env']} (不會儲存)", type="password")
if api_key:
    os.environ[provider_options[provider]["env"]] = api_key.strip()

```

這段的意思是：

- 在 Streamlit 網頁上顯示一個輸入框
- 你在網頁上輸入金鑰後，它會塞到 `api_key` 變數，再寫進對應的環境變數（例如 `OPENAI_API_KEY`）

👉 所以正常用法是不需要改程式，而是：

1. `streamlit run xxx.py` 開起來。
2. 在網頁上看到「Lucky Vicky API 設定」那一塊：
 - 選好「模型供應商」（例如 `openai`）。
 - 下面會出現輸入框，標題類似 `OPENAI_API_KEY (不會儲存)`。
3. 在這個輸入框裡貼上你的金鑰 → 就是 `api_key` 要放的地方。

如果你真的想「寫死」在程式裡（只建議本機測試用，千萬不要 commit 到 GitHub）可以改成類似：

```

python

st.markdown("#### Lucky Vicky API 設定")
provider_options = {
    "groq": {
        "env": "GROQ_API_KEY",

```

```

        "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
    },
    "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
    "mistral": {"env": "MISTRAL_API_KEY", "models": ["ministral-8b-latest", "mistral-large-latest"]},
}

provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])

# ! 只示範，實務上請用環境變數或 .env，不要硬寫在程式
HARDCODED_KEYS = {
    "openai": "h97MT06BhbTrKUqDzEq4zjVenhpc32Bm", # 改成你「新產生」的 key
    # "groq": "你的 groq key",
    # "mistral": "你的 mistral key",
}

api_key = HARDCODED_KEYS.get(provider, "")

if api_key:
    os.environ[provider_options[provider]["env"]] = api_key.strip()
else:
    st.warning("目前這個供應商沒有設定金鑰")

```

但再提醒一次：

- **推薦做法**：把 key 設在系統環境變數，或用 `.env` 搭配 `python-dotenv`，或用你原本的 `st.text_input` 方式。
- **不推薦**：把金鑰硬寫在程式碼裡，尤其是會同步到 GitHub 或給別人看的專案。

如果你跟我說這組 key 是哪一個服務的（OpenAI / Groq / Mistral），我可以幫你寫一個最適合那個 provider 的設定方式（用環境變數那種安全版）。



重新載入測試題庫

```

from __future__ import annotations

import glob
import io
import os
import random
import tempfile
import zipfile
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image

```

```

try:
    import cv2
except ImportError:
    cv2 = None

try:
    from insightface.app import FaceAnalysis
except ImportError:
    FaceAnalysis = None

try:
    import aisuite as ai
except ImportError:
    ai = None

st.set_page_config(page_title="IVE PK + Lucky Vicky Hub", layout="wide")

@st.cache_resource(show_spinner=False)
def load_face_model() -> Optional[FaceAnalysis]:
    """Load InsightFace FaceAnalysis model once per session."""
    if FaceAnalysis is None:
        return None

    app = FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])
    app.prepare(ctx_id=0, det_size=(640, 640))
    return app

def ensure_folder_structure(members: List[str], base_dirs: List[str]) -> List[str]:
    created = []
    for base in base_dirs:
        for member in members:
            path = os.path.join(base, member)
            os.makedirs(path, exist_ok=True)
            created.append(path)
    return created

def iter_images(folder: str) -> List[str]:
    return [
        path
        for path in glob.glob(os.path.join(folder, "*"))
        if path.lower().endswith((".jpg", ".jpeg", ".png", ".bmp"))
    ]

def summarize_folder(folder: str, members: List[str], member_dict: Dict[str, str]) -> pd.DataFrame:
    rows = []
    for member in members:

```

```

path = os.path.join(folder, member)
count = len(iter_images(path)) if os.path.isdir(path) else 0
rows.append(
    {
        "英文名稱": member,
        "顯示名稱": member_dict.get(member, ""),
        "照片數": count,
        "路徑": os.path.abspath(path),
    }
)
return pd.DataFrame(rows)

```

```

def extract_face_embedding(app: FaceAnalysis, image_path: str) -> Optional[np.ndarray]:
    if app is None or cv2 is None:
        return None

    img = cv2.imread(image_path)
    if img is None:
        return None

    faces = app.get(img)
    if not faces:
        return None

    return faces[0].embedding

```

```

def build_face_database(
    app: FaceAnalysis,
    photos_dir: str,
    members: List[str],
) -> Tuple[Dict[str, np.ndarray], pd.DataFrame]:
    stats = []
    database: Dict[str, np.ndarray] = {}

    for member in members:
        folder = os.path.join(photos_dir, member)
        image_files = iter_images(folder)
        embeddings = []

        for path in image_files:
            vec = extract_face_embedding(app, path)
            if vec is not None:
                embeddings.append(vec)

        if embeddings:
            database[member] = np.mean(embeddings, axis=0)

    stats.append(

```



```

    {
        "英文名稱": member,
        "訓練照片": len(image_files),
        "成功提取": len(embeddings),
    }
)

return database, pd.DataFrame(stats)

def recognize_member(
    face_db: Dict[str, np.ndarray],
    features: Optional[np.ndarray],
) -> Tuple[Optional[str], float]:
    if not face_db or features is None:
        return None, 0.0

    best_match = None
    best_score = float("inf")

    for member, db_vec in face_db.items():
        similarity = np.dot(features, db_vec) / (np.linalg.norm(features) * np.linalg.norm(db_vec))
        distance = 1 - similarity
        if distance < best_score:
            best_score = distance
            best_match = member

    confidence = 1 - best_score
    return best_match, confidence

def evaluate_testset(
    app: FaceAnalysis,
    face_db: Dict[str, np.ndarray],
    test_dir: str,
    members: List[str],
    member_dict: Dict[str, str],
    threshold: float,
) -> pd.DataFrame:
    rows = []

    for member in members:
        folder = os.path.join(test_dir, member)
        for path in iter_images(folder):
            features = extract_face_embedding(app, path)
            prediction, confidence = recognize_member(face_db, features)
            predicted_zh = member_dict.get(prediction or "", "無法辨識")
            actual_zh = member_dict.get(member, member)
            is_correct = bool(prediction == member and confidence >= threshold)

```

```

rows.append(
    {
        "圖片": os.path.basename(path),
        "實際成員": actual_zh,
        "AI 預測": predicted_zh if confidence >= threshold else "信心不足",
        "信心度": round(confidence, 3),
        "結果": "✅ 正確" if is_correct else "❌ 錯誤",
        "來源": os.path.abspath(path),
    }
)

return pd.DataFrame(rows)

def unzip_dataset(upload, target_dir: str) -> None:
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp.write(upload.read())
        tmp_path = tmp.name

    with zipfile.ZipFile(tmp_path, "r") as zip_ref:
        zip_ref.extractall(target_dir)

    os.remove(tmp_path)

@dataclass
class QuizState:
    pool: List[Dict[str, str]]
    current: Optional[Dict[str, str]]
    user_score: int = 0
    ai_score: int = 0
    rounds: int = 0
    last_result: str = ""

def prepare_quiz_pool(test_dir: str, members: List[str], member_dict: Dict[str, str]) -> List[Dict[str, str]]:
    pool = []
    for member in members:
        folder = os.path.join(test_dir, member)
        for path in iter_images(folder):
            pool.append(
                {
                    "path": path,
                    "member_en": member,
                    "member_zh": member_dict.get(member, member),
                }
            )
    random.shuffle(pool)
    return pool

```

```
def lucky_vicky_post(
    system_prompt: str,
    user_prompt: str,
    provider: str,
    model: str,
) -> str:
    if ai is None:
        raise RuntimeError("尚未安裝 aisuite，請先執行 pip install aisuite[all]。")

    client = ai.Client()
    response = client.chat.completions.create(
        model=f"{provider}:{model}",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content


def require_packages() -> bool:
    missing = []
    if FaceAnalysis is None:
        missing.append("insightface")
    if cv2 is None:
        missing.append("opencv-python")

    if missing:
        st.warning(
            f"缺少套件：{'，'.join(missing)}。請先執行 "
            "pip install insightface onnxruntime opencv-python 後再啟動 Streamlit。 "
        )
    return False
return True


def main() -> None:
    st.title("IVE 成員 PK + Lucky Vicky Hub")
    st.caption("結合 DeepFace/InsightFace PK 與 AISuite Lucky Vicky 生成器的一站式 Streamlit 介面。")

    default_en = "yujin,wonyoung,gaeul,rei,liz,leeseo"
    default_zh = "兪真유진,員瑛원영,秋天가을,Rei레이,Liz리즈,李瑞이서"

    with st.sidebar:
        st.header("基本設定")
        members_en_str = st.text_input("成員英文名稱 (逗號隔開)", default_en)
        members_zh_str = st.text_input("成員顯示名稱 (逗號隔開)", default_zh)
```

```

photos_dir = st.text_input("訓練照片資料夾", "photos")
test_dir = st.text_input("測試照片資料夾", "test_photos")
threshold = st.slider("AI 信心門檻", min_value=0.3, max_value=0.95, value=0.6, step=0.05)
max_rounds = st.number_input("PK 回合數", min_value=3, max_value=30, value=10, step=1)

st.divider()
st.markdown("#### Lucky Vicky API 設定")
provider_options = {
    "mistral": {"env": "MISTRAL_API_KEY", "models": ["mistral-8b-latest", "mistral-large-latest"]},
    "groq": {
        "env": "GROQ_API_KEY",
        "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
    },
    "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
}
provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])
HARDCODED_KEYS = {
    # "openai": " ",
    # "groq": " ",
    "mistral": "h97MT06BhbTrKUqDzEq4zjVenhpc32Bm",
}

api_key = HARDCODED_KEYS.get(provider, "")
if api_key:
    os.environ[provider_options[provider]["env"]] = api_key.strip()
    st.caption(f"已載入預設金鑰：{provider} -> {api_key}")
else:
    st.warning("目前這個供應商沒有設定金鑰，請使用 Hardcoded Keys。")

members_en = [m.strip() for m in members_en_str.split(",") if m.strip()]
members_zh = [m.strip() for m in members_zh_str.split(",") if m.strip()]

if len(members_en) != len(members_zh):
    st.error("英文名稱與顯示名稱數量不一致，請重新確認設定。")
    return

member_dict = dict(zip(members_en, members_zh))

if "face_db" not in st.session_state:
    st.session_state["face_db"] = {}
if "db_stats" not in st.session_state:
    st.session_state["db_stats"] = pd.DataFrame()
if "test_results" not in st.session_state:
    st.session_state["test_results"] = pd.DataFrame()
if "quiz_state" not in st.session_state:
    st.session_state["quiz_state"] = QuizState(pool=[], current=None)

require_packages()
model_app = load_face_model()

```

```

tabs = st.tabs(
    [
        "1 資料夾管理",
        "2 人臉特徵 & 測試",
        "3 IVE PK 小遊戲",
        "4 Lucky Vicky 生成器",
    ]
)

# Tab 1: dataset helper
with tabs[0]:
    st.subheader("資料夾管理與資料準備")
    st.markdown(
        """
        1. 建立 photos 與 test_photos 的成員子資料夾。
        2. 可直接上傳 zip (例如 Demo03v 的 photos.zip) 並解壓縮到指定資料夾。
        3. 下方會顯示當前資料夾的照片數量統計。
        """
    )

    col1, col2 = st.columns(2)
    with col1:
        if st.button("建立資料夾結構"):
            ensure_folder_structure(members_en, [photos_dir, test_dir])
            st.success("資料夾建立完成，請開始放入照片。")

    with col2:
        upload = st.file_uploader("上傳 photos/test_photos zip", type="zip")
        if upload is not None:
            unzip_dataset(upload, ".")
            st.success("解壓縮完成，請到下方查看統計。")

    st.markdown("##### 訓練照片概況")
    train_df = summarize_folder(photos_dir, members_en, member_dict)
    st.dataframe(train_df, use_container_width=True)

    st.markdown("##### 測試照片概況")
    test_df = summarize_folder(test_dir, members_en, member_dict)
    st.dataframe(test_df, use_container_width=True)

# Tab 2: feature building + evaluation
with tabs[1]:
    st.subheader("InsightFace 特徵建立與測試")
    if model_app is None:
        st.warning("尚未載入 InsightFace 模型，請確認必要套件已安裝。")
    else:
        if st.button("重新計算人臉特徵資料庫"):
            with st.spinner("建立資料庫中..."):

```

```

face_db, stats = build_face_database(model_app, photos_dir, members_en)
st.session_state["face_db"] = face_db
st.session_state["db_stats"] = stats
st.success(f"完成！成功建立 {len(face_db)} 位成員的特徵。")

if not st.session_state["face_db"]:
    st.info("請先建立人臉特徵資料庫。")
else:
    st.markdown("##### 特徵建立結果")
    st.dataframe(st.session_state["db_stats"], use_container_width=True)

if st.button("測試 test_photos 準確率"):
    with st.spinner("評估中..."):
        results = evaluate_testset(
            model_app,
            st.session_state["face_db"],
            test_dir,
            members_en,
            member_dict,
            threshold,
        )
        st.session_state["test_results"] = results

if not st.session_state["test_results"].empty:
    results = st.session_state["test_results"]
    st.markdown(f"##### 測試結果，共 {len(results)} 張")
    correct = (results["結果"] == "✅ 正確").sum()
    accuracy = correct / len(results) * 100 if len(results) else 0
    st.metric("準確率", f"{accuracy:.1f}%")
    st.dataframe(results, use_container_width=True)

st.divider()
st.markdown("##### 即時單張測試")
test_image = st.file_uploader("上傳單張照片進行辨識", type=["jpg", "jpeg", "png"],
key="single_test")
if test_image is not None:
    image_bytes = test_image.read()
    image = Image.open(io.BytesIO(image_bytes))
    st.image(image, caption="上傳圖片", width=300)

    temp_path = os.path.join(tempfile.gettempdir(), f"streamlit_test_{test_image.name}")
    with open(temp_path, "wb") as f:
        f.write(image_bytes)

    features = extract_face_embedding(model_app, temp_path)
    prediction, conf = recognize_member(st.session_state["face_db"], features)
    predicted_name = member_dict.get(prediction or "", "無法辨識")

    if prediction and conf >= threshold:
        st.success(f"AI 預測：{predicted_name} (信心度 {conf:.2f})")

```

```

else:
    st.error("無法辨識，請嘗試清晰的單人照片。")

# Tab 3: PK mini-game
with tabs[2]:
    st.subheader("和 AI PK 誰比較會認 IVE 成員")
    quiz_state: QuizState = st.session_state["quiz_state"]

    col1, col2, col3 = st.columns(3)
    with col1:
        if st.button("重新載入測試題庫"):
            quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
            quiz_state.current = None
            quiz_state.user_score = 0
            quiz_state.ai_score = 0
            quiz_state.rounds = 0
            quiz_state.last_result = ""
            st.toast(f"載入 {len(quiz_state.pool)} 題完成！")

    with col2:
        if st.button("下一題", disabled=not quiz_state.pool):
            quiz_state.current = random.choice(quiz_state.pool)
            st.session_state["user_choice"] = ""

    with col3:
        if st.button("重置分數"):
            quiz_state.user_score = 0
            quiz_state.ai_score = 0
            quiz_state.rounds = 0
            quiz_state.last_result = ""

    if not st.session_state["face_db"]:
        st.info("請先建立人臉特徵資料庫，AI 才能參與 PK。")
    elif not quiz_state.pool:
        st.info("請先載入測試題庫。")
    else:
        options = list(member_dict.values())
        if not options:
            st.warning("沒有可選的成員名稱，請先在側欄設定成員名單。")
        elif quiz_state.current:
            st.image(quiz_state.current["path"], caption="請猜猜這是哪位成員？", width=320)

            if (
                "user_choice" in st.session_state
                and st.session_state["user_choice"] not in options
            ):
                del st.session_state["user_choice"]

            with st.form("pk_form"):
                user_choice = st.radio(

```

```

        "你的答案",
        options=options,
        key="user_choice",
        horizontal=True,
    )
    submitted = st.form_submit_button("提交答案")

    if submitted and user_choice:
        features = extract_face_embedding(model_app, quiz_state.current["path"])
        prediction, conf = recognize_member(st.session_state["face_db"], features)
        ai_choice = member_dict.get(prediction or "", "無法辨識")
        correct_answer = quiz_state.current["member_zh"]

        user_correct = user_choice == correct_answer
        ai_correct = prediction == quiz_state.current["member_en"] and conf >= threshold

        quiz_state.rounds += 1
        quiz_state.user_score += int(user_correct)
        quiz_state.ai_score += int(ai_correct)

        quiz_state.last_result = (
            f"第 {quiz_state.rounds} 回合：正解 {correct_answer}\n"
            f"你回答 {user_choice} {'✅' if user_correct else '❌'}；"
            f"AI 回答 {ai_choice} (信心 {conf:.2f}) {'✅' if ai_correct else '❌'}"
        )

        if quiz_state.rounds >= max_rounds:
            if quiz_state.user_score > quiz_state.ai_score:
                quiz_state.last_result += "\n🏆 你戰勝 AI !"
            elif quiz_state.user_score < quiz_state.ai_score:
                quiz_state.last_result += "\n🤖 AI 更勝一籌，下次加油！"
            else:
                quiz_state.last_result += "\n👉 平手，勢均力敵！"

    st.markdown(
        f"""
        **目前比分**
        👤 玩家：{quiz_state.user_score} 分
        🤖 AI：{quiz_state.ai_score} 分
        已玩：{quiz_state.rounds}/{max_rounds} 回合
        """
    )

    if quiz_state.last_result:
        st.info(quiz_state.last_result)

# Tab 4: Lucky Vicky generator
with tabs[3]:
    st.subheader("Lucky Vicky 員瑛式思考生成器 ")
    system_prompt = st.text_area(

```



```

"System Prompt",
value=(
    "請用台灣習慣的中文來寫這段 po 文：\n"
    "請用員瑛式思考, 什麼都正向思維任何使用者的事情，"
    "用第一人稱社群媒體口吻說一次，說明為什麼這是一件超幸運的事，"
    "並且以「完全是 Lucky Vicky 呀!」結尾，可以加入 emoji。"
),
height=160,
)
user_prompt = st.text_area("請輸入想轉念的事件", height=120, placeholder="例如：今天咖啡灑到電腦上了...")

if st.button("Lucky Vicky 魔法 ✨"):
    if not api_key:
        st.error("請先在左側輸入 API 金鑰。")
    elif not user_prompt.strip():
        st.error("請輸入事件描述。")
    else:
        try:
            with st.spinner("生成中..."):
                result = lucky_vicky_post(system_prompt, user_prompt, provider, model)
            st.success("生成完成！")
            st.write(result)
        except Exception as exc: # noqa: BLE001
            st.error(f"呼叫 LLM 時發生錯誤：{exc}")

if __name__ == "__main__":
    main()

```

請問他載入的題庫是什麼資料夾

它載入的題庫就是你在側欄填的「**測試照片資料夾**」下面的各成員子資料夾。

看這段：

```

python

# 側欄
test_dir = st.text_input("測試照片資料夾", "test_photos") # 預設是 "test_photos"
...
if st.button("重新載入測試題庫"):
    quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)

```

`prepare_quiz_pool` 裡面又是這樣抓圖：

```
python
```

```
for member in members:
    folder = os.path.join(test_dir, member) # 例如 test_photos/yujin
    for path in iter_images(folder):
        pool.append({...})
```

所以題庫來源是：

{測試照片資料夾}/{成員英文名稱}/裡面的所有圖片

用預設設定的話，就是：

- ./test_photos/yujin/
- ./test_photos/wonyoung/
- ./test_photos/gaeul/
- ./test_photos/rei/
- ./test_photos/liz/
- ./test_photos/leeseo/

底下的 JPG/PNG 等圖片都會被當成「測試題庫」，按「重新載入測試題庫」時就從這些資料夾重新掃一次。如果你在側欄把「測試照片資料夾」改成別的名稱，例如 `my_test`，那題庫就會變成 `./my_test/{成員英文}/`。



test_photos 要改成 photo 怎麼改

有兩種改法，看你想「只改這次」還是「以後都用 photo」。

1 只改這次（不用改程式）

跑起 Streamlit 之後，在左邊側欄：

- 找到那一格：測試照片資料夾
- 原本裡面是 `test_photos`
- 把它改成 `photo`，按 Enter 或點別的地方即可
- 然後按「重新載入測試題庫」按鈕

這樣程式就會去讀 `./photo/成員英文名稱/` 底下的圖片了。

2 以後預設就用 photo（改程式碼）

把這行：

```
python

test_dir = st.text_input("測試照片資料夾", "test_photos")
```

改成：

```
python

test_dir = st.text_input("測試照片資料夾", "photo")
```

存檔後重新跑 `streamlit run xxx.py`，預設就會是 `photo`。

記得在專案資料夾裡建立：

`photo/yujin`, `photo/wonyoung`, ... 這些子資料夾，把測試用照片放進去，
這樣 PK 題庫才有東西可以載入。



我是要改 IVE PK 這邊小遊戲的題庫

好，我懂你要的是「**只有 IVE PK 小遊戲**」的題庫改成用 `photo` 資料夾，而不是整個程式的 `test_photos` 都改。
PK 小遊戲現在的題庫是這一行決定的：

```
python

if st.button("重新載入測試題庫"):
    quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
```

這裡用的是 `test_dir`，也就是側欄「測試照片資料夾」（預設 `test_photos`）。



做法一：PK 題庫改用 `photos`（訓練資料夾）

如果你想讓 PK 題庫直接用訓練的 `photos`：

1. 找到 Tab 3（IVE PK 小遊戲）裡這段：

```
python

with coll:
    if st.button("重新載入測試題庫"):
        quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
```

2. 把 `test_dir` 改成 `photos_dir`（就是上面側欄那個「訓練照片資料夾」變數）：

```
python

with coll:
    if st.button("重新載入測試題庫"):
        quiz_state.pool = prepare_quiz_pool(photos_dir, members_en, member_dict)
```

這樣：

- **Tab 2 測試準確率** 還是用 `test_dir` (`test_photos`)
- **Tab 3 PK 小遊戲** 題庫就改成 `photos_dir` (`photos`)

✓ 做法二：PK 題庫專用 photo 這個資料夾

如果你想 PK 題庫用一個獨立資料夾叫 `photo`，可以這樣改：

1. 在側欄多一個欄位

在側欄那段加一行（放在 `test_dir` 下面即可）：

```
python

photos_dir = st.text_input("訓練照片資料夾", "photos")
test_dir = st.text_input("測試照片資料夾", "test_photos")
pk_dir = st.text_input("PK 題庫資料夾", "photo") # NEW 新增這一行
```

2. PK 小遊戲改用 pk_dir

把原來這行：

```
python

quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
```

改成：

```
python

quiz_state.pool = prepare_quiz_pool(pk_dir, members_en, member_dict)
```

然後你在側欄把「PK 題庫資料夾」填成 `photo`，實際的圖檔就放在：

- `photo/yujin/`
- `photo/wonyoung/`
- `photo/gaeul/`
- ...

按「重新載入測試題庫」後，PK 遊戲就會從 `photo` 這個資料夾底下出題。

如果你把你現在想用的資料夾結構跟我說（例如：`./photo/成員英文/` 已經有圖了？還是全部都在 `./photos`？），我可以直接幫你改成一版完整、貼上就能跑的程式。



```
from __future__ import annotations

import glob
import io
import os
import random
import tempfile
import zipfile
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image

try:
    import cv2
except ImportError:
    cv2 = None

try:
    from insightface.app import FaceAnalysis
except ImportError:
    FaceAnalysis = None

try:
    import aisuite as ai
except ImportError:
    ai = None

st.set_page_config(page_title="IVE PK + Lucky Vicky Hub", layout="wide")

@st.cache_resource(show_spinner=False)
def load_face_model() -> Optional[FaceAnalysis]:
    """Load InsightFace FaceAnalysis model once per session."""
    if FaceAnalysis is None:
        return None

    app = FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])
    app.prepare(ctx_id=0, det_size=(640, 640))
    return app
```

```
def ensure_folder_structure(members: List[str], base_dirs: List[str]) -> List[str]:
    created = []
    for base in base_dirs:
        for member in members:
            path = os.path.join(base, member)
            os.makedirs(path, exist_ok=True)
            created.append(path)
    return created

def iter_images(folder: str) -> List[str]:
    return [
        path
        for path in glob.glob(os.path.join(folder, "*"))
        if path.lower().endswith((".jpg", ".jpeg", ".png", ".bmp"))
    ]

def summarize_folder(folder: str, members: List[str], member_dict: Dict[str, str]) -> pd.DataFrame:
    rows = []
    for member in members:
        path = os.path.join(folder, member)
        count = len(iter_images(path)) if os.path.isdir(path) else 0
        rows.append(
            {
                "英文名稱": member,
                "顯示名稱": member_dict.get(member, ""),
                "照片數": count,
                "路徑": os.path.abspath(path),
            }
        )
    return pd.DataFrame(rows)

def sample_member_images(
    base_dir: str,
    members: List[str],
    member_dict: Dict[str, str],
    per_member: int = 1,
    limit: int = 12,
) -> List[Dict[str, str]]:
    """Pick a few images per member so the UI can show quick previews."""
    samples: List[Dict[str, str]] = []
    for member in members:
        folder = os.path.join(base_dir, member)
        candidates = iter_images(folder)
        if not candidates:
            continue
        picks = random.sample(candidates, k=min(per_member, len(candidates)))
```

```

for path in picks:
    samples.append(
        {
            "path": path,
            "member_en": member,
            "member_zh": member_dict.get(member, member),
        }
    )
random.shuffle(samples)
return samples[:limit]

```

```

def extract_face_embedding(app: FaceAnalysis, image_path: str) -> Optional[np.ndarray]:
    if app is None or cv2 is None:
        return None

    img = cv2.imread(image_path)
    if img is None:
        return None

    faces = app.get(img)
    if not faces:
        return None

    return faces[0].embedding

```

```

def build_face_database(
    app: FaceAnalysis,
    photos_dir: str,
    members: List[str],
) -> Tuple[Dict[str, np.ndarray], pd.DataFrame]:
    stats = []
    database: Dict[str, np.ndarray] = {}

    for member in members:
        folder = os.path.join(photos_dir, member)
        image_files = iter_images(folder)
        embeddings = []

        for path in image_files:
            vec = extract_face_embedding(app, path)
            if vec is not None:
                embeddings.append(vec)

        if embeddings:
            database[member] = np.mean(embeddings, axis=0)

    stats.append(
        {
            "英文名稱": member,

```

```

        "訓練照片": len(image_files),
        "成功提取": len(embeddings),
    }
)

return database, pd.DataFrame(stats)

def recognize_member(
    face_db: Dict[str, np.ndarray],
    features: Optional[np.ndarray],
) -> Tuple[Optional[str], float]:
    if not face_db or features is None:
        return None, 0.0

    best_match = None
    best_score = float("inf")

    for member, db_vec in face_db.items():
        denom = float(np.linalg.norm(features) * np.linalg.norm(db_vec))
        if denom == 0:
            continue
        similarity = float(np.dot(features, db_vec) / denom)
        distance = 1 - similarity
        if distance < best_score:
            best_score = distance
            best_match = member

    confidence = 1 - best_score
    return best_match, confidence

def rank_members(
    face_db: Dict[str, np.ndarray],
    features: Optional[np.ndarray],
    top_k: int = 3,
) -> List[Tuple[str, float]]:
    if not face_db or features is None:
        return []

    scores: List[Tuple[str, float]] = []
    for member, db_vec in face_db.items():
        denom = float(np.linalg.norm(features) * np.linalg.norm(db_vec))
        if denom == 0:
            continue
        similarity = float(np.dot(features, db_vec) / denom)
        scores.append((member, similarity))

    # Sort by confidence descending
    scores.sort(key=lambda item: item[1], reverse=True)
    return scores[:top_k]

```



```
def evaluate_testset(
    app: FaceAnalysis,
    face_db: Dict[str, np.ndarray],
    test_dir: str,
    members: List[str],
    member_dict: Dict[str, str],
    threshold: float,
) -> pd.DataFrame:
    rows = []

    for member in members:
        folder = os.path.join(test_dir, member)
        for path in iter_images(folder):
            features = extract_face_embedding(app, path)
            prediction, confidence = recognize_member(face_db, features)
            predicted_zh = member_dict.get(prediction or "", "無法辨識")
            actual_zh = member_dict.get(member, member)
            is_correct = bool(prediction == member and confidence >= threshold)

            rows.append(
                {
                    "圖片": os.path.basename(path),
                    "實際成員": actual_zh,
                    "AI 預測": predicted_zh if confidence >= threshold else "信心不足",
                    "信心度": round(confidence, 3),
                    "結果": "✅ 正確" if is_correct else "❌ 錯誤",
                    "來源": os.path.abspath(path),
                }
            )

    return pd.DataFrame(rows)

def summarize_accuracy(results: pd.DataFrame) -> pd.DataFrame:
    if results.empty:
        return pd.DataFrame()

    temp = results.copy()
    temp["正確"] = temp["結果"] == "✅ 正確"
    summary = temp.groupby("實際成員").agg(
        總張數=("圖片", "count"),
        答對張數=("正確", "sum"),
        平均信心=("信心度", "mean"),
    )
    summary["準確率(%)"] = (summary["答對張數"] / summary["總張數"] * 100).round(1)
    summary["平均信心"] = summary["平均信心"].round(3)
    return summary.reset_index()
```

```
def unzip_dataset(upload, target_dir: str) -> None:
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp.write(upload.read())
        tmp_path = tmp.name

    with zipfile.ZipFile(tmp_path, "r") as zip_ref:
        zip_ref.extractall(target_dir)

    os.remove(tmp_path)

@dataclass
class QuizState:
    pool: List[Dict[str, str]]
    current: Optional[Dict[str, str]]
    user_score: int = 0
    ai_score: int = 0
    rounds: int = 0
    last_result: str = ""

def prepare_quiz_pool(test_dir: str, members: List[str], member_dict: Dict[str, str]) -> List[Dict[str, str]]:
    pool = []
    for member in members:
        folder = os.path.join(test_dir, member)
        for path in iter_images(folder):
            pool.append(
                {
                    "path": path,
                    "member_en": member,
                    "member_zh": member_dict.get(member, member),
                }
            )
    random.shuffle(pool)
    return pool

def lucky_vicky_post(
    system_prompt: str,
    user_prompt: str,
    provider: str,
    model: str,
) -> str:
    if ai is None:
        raise RuntimeError("尚未安裝 aisuite，請先執行 pip install aisuite[all]。")

    client = ai.Client()
    response = client.chat.completions.create(
```

```

model=f"{provider}:{model}",
messages=[
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_prompt},
],
)
return response.choices[0].message.content

def require_packages() -> bool:
    missing = []
    if FaceAnalysis is None:
        missing.append("insightface")
    if cv2 is None:
        missing.append("opencv-python")

    if missing:
        st.warning(
            f"缺少套件：{'', '.join(missing)}。請先執行 "
            "pip install insightface onnxruntime opencv-python 後再啟動 Streamlit。 "
        )
        return False
    return True

def main() -> None:
    st.title("IVE 成員 PK + Lucky Vicky Hub")
    st.caption("結合 DeepFace/InsightFace PK 與 AISuite Lucky Vicky 生成器的一站式 Streamlit 介面。")

    default_en = "yujin,wonyoung,gaeul,rei,liz,leeseo"
    default_zh = "兪真유진,員瑛원영,秋天가을,Rei레이,Liz리즈,李瑞이서"

    with st.sidebar:
        st.header("基本設定")
        members_en_str = st.text_input("成員英文名稱 (逗號隔開)", default_en)
        members_zh_str = st.text_input("成員顯示名稱 (逗號隔開)", default_zh)
        photos_dir = st.text_input("訓練照片資料夾", "photos")
        test_dir = st.text_input("測試照片資料夾", "test_photos")
        threshold = st.slider("AI 信心門檻", min_value=0.3, max_value=0.95, value=0.6, step=0.05)
        max_rounds = st.number_input("PK 回合數", min_value=3, max_value=30, value=10, step=1)

    st.divider()
    st.markdown("#### Lucky Vicky API 設定")
    provider_options = {
        "mistral": {"env": "MISTRAL_API_KEY", "models": ["ministral-8b-latest", "mistral-large-latest"]},
        "groq": {
            "env": "GROQ_API_KEY",
            "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
        }
    }

```

```

    },
    "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
}
provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])
HARDCODED_KEYS = {
    # "openai": " ",
    # "groq": " ",
    "mistral": "h97MT06BhbTrKUqDzEq4zjVenhpc32Bm",
}

api_key = HARDCODED_KEYS.get(provider, "")
if api_key:
    os.environ[provider_options[provider]["env"]] = api_key.strip()
    st.caption(f"已載入預設金鑰：{provider} -> {api_key}")
else:
    st.warning("目前這個供應商沒有設定金鑰，請使用 Hardcoded Keys。")

members_en = [m.strip() for m in members_en_str.split(",") if m.strip()]
members_zh = [m.strip() for m in members_zh_str.split(",") if m.strip()]

if len(members_en) != len(members_zh):
    st.error("英文名稱與顯示名稱數量不一致，請重新確認設定。")
    return

member_dict = dict(zip(members_en, members_zh))

if "face_db" not in st.session_state:
    st.session_state["face_db"] = {}
if "db_stats" not in st.session_state:
    st.session_state["db_stats"] = pd.DataFrame()
if "test_results" not in st.session_state:
    st.session_state["test_results"] = pd.DataFrame()
if "quiz_state" not in st.session_state:
    st.session_state["quiz_state"] = QuizState(pool=[], current=None)

require_packages()
model_app = load_face_model()

tabs = st.tabs(
    [
        "1 資料夾管理",
        "2 人臉特徵 & 測試",
        "3 IVE PK 小遊戲",
        "4 Lucky Vicky 生成器",
    ]
)

# Tab 1: dataset helper
with tabs[0]:

```

```

st.subheader("資料夾管理與資料準備")
st.markdown(
    """
    1. 建立 photos 與 test_photos 的成員子資料夾。
    2. 可直接上傳 zip (例如 Demo03v 的 photos.zip) 並解壓縮到指定資料夾。
    3. 下方會顯示當前資料夾的照片數量統計。
    """
)

col1, col2 = st.columns(2)
with col1:
    if st.button("建立資料夾結構"):
        ensure_folder_structure(members_en, [photos_dir, test_dir])
        st.success("資料夾建立完成，請開始放入照片。")

with col2:
    upload = st.file_uploader("上傳 photos/test_photos zip", type="zip")
    if upload is not None:
        unzip_dataset(upload, ".")
        st.success("解壓縮完成，請到下方查看統計。")

st.markdown("##### 訓練照片概況")
train_df = summarize_folder(photos_dir, members_en, member_dict)
st.dataframe(train_df, use_container_width=True)

st.markdown("##### 測試照片概況")
test_df = summarize_folder(test_dir, members_en, member_dict)
st.dataframe(test_df, use_container_width=True)

st.markdown("##### 照片快速檢視")
with st.expander("隨機抽樣，檢查資料品質"):
    sample_from = st.radio("抽樣來源", ["訓練 photos", "測試 test_photos"], horizontal=True)
    per_member = st.slider("每位成員抽幾張？", 0, 3, 1, step=1)
    total_limit = st.slider("最多顯示張數", 3, 18, 9, step=3)
    if st.button("抽樣預覽", type="primary"):
        base_dir = photos_dir if sample_from == "訓練 photos" else test_dir
        samples = sample_member_images(base_dir, members_en, member_dict,
        per_member=per_member, limit=total_limit)
        if not samples:
            st.info("目前沒有可抽樣的照片，請先放入影像。")
        else:
            for i in range(0, len(samples), 3):
                cols = st.columns(3)
                for col, sample in zip(cols, samples[i : i + 3]):
                    caption = f"{sample['member_zh']} | {os.path.basename(sample['path'])}"
                    col.image(sample["path"], caption=caption, use_container_width=True)

# Tab 2: feature building + evaluation
with tabs[1]:
    st.subheader("InsightFace 特徵建立與測試")

```

```

if model_app is None:
    st.warning("尚未載入 InsightFace 模型，請確認必要套件已安裝。")
else:
    if st.button("重新計算人臉特徵資料庫"):
        with st.spinner("建立資料庫中..."):
            face_db, stats = build_face_database(model_app, photos_dir, members_en)
            st.session_state["face_db"] = face_db
            st.session_state["db_stats"] = stats
            st.success(f"完成！成功建立 {len(face_db)} 位成員的特徵。")

    if not st.session_state["face_db"]:
        st.info("請先建立人臉特徵資料庫。")
    else:
        st.markdown("##### 特徵建立結果")
        st.dataframe(st.session_state["db_stats"], use_container_width=True)

    if st.button("測試 test_photos 準確率"):
        with st.spinner("評估中..."):
            results = evaluate_testset(
                model_app,
                st.session_state["face_db"],
                test_dir,
                members_en,
                member_dict,
                threshold,
            )
            st.session_state["test_results"] = results

    if not st.session_state["test_results"].empty:
        results = st.session_state["test_results"]
        st.markdown(f"##### 測試結果，共 {len(results)} 張")
        correct = (results["結果"] == "✅ 正確").sum()
        accuracy = correct / len(results) * 100 if len(results) else 0
        st.metric("準確率", f"{accuracy:.1f}%")
        st.dataframe(results, use_container_width=True)
        st.download_button(
            "下載逐張結果 (CSV)",
            results.to_csv(index=False).encode("utf-8"),
            file_name="test_results.csv",
            mime="text/csv",
        )

    summary = summarize_accuracy(results)
    if not summary.empty:
        st.markdown("##### 各成員表現")
        summary = summary.sort_values(by="準確率(%)", ascending=True)
        st.dataframe(summary, use_container_width=True)
        st.download_button(
            "下載成員摘要 (CSV)",

```

```

summary.to_csv(index=False).encode("utf-8"),
file_name="member_summary.csv",
mime="text/csv",
)

st.divider()
st.markdown("##### 即時單張測試")
test_image = st.file_uploader("上傳單張照片進行辨識", type=["jpg", "jpeg", "png"],
key="single_test")
if test_image is not None:
    image_bytes = test_image.read()
    image = Image.open(io.BytesIO(image_bytes))
    st.image(image, caption="上傳圖片", width=300)

    temp_path = os.path.join(tempfile.gettempdir(), f"streamlit_test_{test_image.name}")
    with open(temp_path, "wb") as f:
        f.write(image_bytes)

    features = extract_face_embedding(model_app, temp_path)
    prediction, conf = recognize_member(st.session_state["face_db"], features)
    predicted_name = member_dict.get(prediction or "", "無法辨識")

    if prediction and conf >= threshold:
        st.success(f"AI 預測：{predicted_name} (信心度 {conf:.2f})")
    else:
        st.error("無法辨識，請嘗試清晰的單人照片。")

# Tab 3: PK mini-game
with tabs[2]:
    st.subheader("和 AI PK 誰比較會認 IVE 成員")
    quiz_state: QuizState = st.session_state["quiz_state"]

    col1, col2, col3 = st.columns(3)
    with col1:
        if st.button("重新載入測試題庫"):
            quiz_state.pool = prepare_quiz_pool(photos_dir, members_en, member_dict)
            quiz_state.current = None
            quiz_state.user_score = 0
            quiz_state.ai_score = 0
            quiz_state.rounds = 0
            quiz_state.last_result = ""
            st.toast(f"載入 {len(quiz_state.pool)} 題完成！")

    with col2:
        if st.button("下一題", disabled=not quiz_state.pool):
            quiz_state.current = random.choice(quiz_state.pool)
            st.session_state["user_choice"] = ""

    with col3:
        if st.button("重置分數"):
```

```

quiz_state.user_score = 0
quiz_state.ai_score = 0
quiz_state.rounds = 0
quiz_state.last_result = ""

if not st.session_state["face_db"]:
    st.info("請先建立人臉特徵資料庫，AI 才能參與 PK。")
elif not quiz_state.pool:
    st.info("請先載入測試題庫。")
else:
    with st.expander("先複習一下各成員長相"):
        study_samples = sample_member_images(photos_dir, members_en, member_dict,
per_member=1, limit=6)
        if not study_samples:
            st.caption("資料夾裡還沒有照片可供複習。")
        else:
            for i in range(0, len(study_samples), 3):
                cols = st.columns(3)
                for col, sample in zip(cols, study_samples[i : i + 3]):
                    col.image(sample["path"], caption=sample["member_zh"],
use_container_width=True)

    options = list(member_dict.values())
    if not options:
        st.warning("沒有可選的成員名稱，請先在側欄設定成員名單。")
    elif quiz_state.current:
        st.image(quiz_state.current["path"], caption="請猜猜這是哪位成員？", width=320)
        with st.expander("需要提示嗎？"):
            if st.button("AI 給我兩個可能的人選"):
                features = extract_face_embedding(model_app, quiz_state.current["path"])
                top_candidates = rank_members(st.session_state["face_db"], features, top_k=2)
                if not top_candidates:
                    st.info("目前無法給提示，可能影像品質不佳。")
                else:
                    for name, conf in top_candidates:
                        st.write(f"{member_dict.get(name, name)} (信心 {conf:.2f})")

    if (
        "user_choice" in st.session_state
        and st.session_state["user_choice"] not in options
    ):
        del st.session_state["user_choice"]

    with st.form("pk_form"):
        user_choice = st.radio(
            "你的答案",
            options=options,
            key="user_choice",
            horizontal=True,
        )

```



```

submitted = st.form_submit_button("提交答案")

if submitted and user_choice:
    features = extract_face_embedding(model_app, quiz_state.current["path"])
    prediction, conf = recognize_member(st.session_state["face_db"], features)
    ai_choice = member_dict.get(prediction or "", "無法辨識")
    correct_answer = quiz_state.current["member_zh"]

    user_correct = user_choice == correct_answer
    ai_correct = prediction == quiz_state.current["member_en"] and conf >= threshold

    quiz_state.rounds += 1
    quiz_state.user_score += int(user_correct)
    quiz_state.ai_score += int(ai_correct)

    quiz_state.last_result = (
        f"第 {quiz_state.rounds} 回合：正解 {correct_answer}\n"
        f"你回答 {user_choice} {'✅' if user_correct else '❌'}；"
        f"AI 回答 {ai_choice} (信心 {conf:.2f}) {'✅' if ai_correct else '❌'}"
    )

    if quiz_state.rounds >= max_rounds:
        if quiz_state.user_score > quiz_state.ai_score:
            quiz_state.last_result += "\n🎉 你戰勝 AI！"
        elif quiz_state.user_score < quiz_state.ai_score:
            quiz_state.last_result += "\n🤖 AI 更勝一籌，下次加油！"
        else:
            quiz_state.last_result += "\n💛 平手，勢均力敵！"

st.markdown(
    f"""
    **目前比分**
    👤 玩家：{quiz_state.user_score} 分
    🤖 AI：{quiz_state.ai_score} 分
    已玩：{quiz_state.rounds}/{max_rounds} 回合
    """
)

if quiz_state.last_result:
    st.info(quiz_state.last_result)

# Tab 4: Lucky Vicky generator
with tabs[3]:
    st.subheader("Lucky Vicky 員瑛式思考生成器 ")
    system_prompt = st.text_area(
        "System Prompt",
        value=(
            "請用台灣習慣的中文來寫這段 po 文：\n"
            "請用員瑛式思考，什麼都正向思維任何使用者的事情，"
            "用第一人稱社群媒體口吻說一次，說明為什麼這是一件超幸運的事，"
        )
    )

```

```

        "並且以「完全是 Lucky Vicky 呀!」結尾，可以加入 emoji。"
    ),
    height=160,
)
user_prompt = st.text_area("請輸入想轉念的事件", height=120, placeholder="例如：今天咖啡灑
到電腦上了...")

if st.button("Lucky Vicky 魔法 ✨"):
    if not api_key:
        st.error("請先在左側輸入 API 金鑰。")
    elif not user_prompt.strip():
        st.error("請輸入事件描述。")
    else:
        try:
            with st.spinner("生成中..."):
                result = lucky_vicky_post(system_prompt, user_prompt, provider, model)
            st.success("生成完成！")
            st.write(result)
        except Exception as exc: # noqa: BLE001
            st.error(f"呼叫 LLM 時發生錯誤：{exc}")

if __name__ == "__main__":
    main()

```

還有可以衍生的空間嗎

有，而且空間其實還不少 😊

你這版已經很完整了（資料夾管理 + 特徵建立 + 評估 + PK + LLM），下面幫你分幾個方向給一些「可以進階延伸」的點，挑你覺得有趣的玩就好。

1 IVE PK 小遊戲：變成「完整遊戲」

(1) 難度模式 (Easy / Normal / Hard)

現在 PK 是：六個成員全選項 + 任意一張 photo。

可以加一個 radio 控制難度，例如：

- **Easy：**
 - 選項只給 3 個成員（正解 + 2 個隨機干擾）
- **Normal：**

- 保持現在的 6 選 1
- **Hard :**
 - 題目只從「AI 信心度低的照片」裡抽（例如 conf 在 0.4~0.7 的）

做法：

- 在側欄加 難度 radio
- `prepare_quiz_pool` 回傳 pool 時順便存一下「這張圖的 AI 信心」，PK 抽題時依難度篩選。

(2) 排行榜 / 多人模式

現在只有單一 `quiz_state`。你可以：

- 讓使用者在側欄輸入「玩家名稱」
- 把每一回合的結果 append 到一個 `results_df`（存在 `st.session_state["pk_log"]`）
- 顯示一個小表格：玩家名 / 玩了幾回 / 平均得分 / 勝過 AI 次數

這樣就可以現場給同學輪流 PK，最後比誰最強。

(3) 題庫標記 + 針對弱點練習

你已經有 `test_results` 了，可以再延伸：

- 將「答錯的題目」收集起來 → 做一個 錯題練習 模式
- 再開一個按鈕：「從我剛剛錯的題目出題」
- 幫使用者加一個「最怕哪一位成員」統計（哪個成員被錯認最多）

2 模型 & 分析：往「作業 / 報告」再升級

(1) 自動尋找最佳 Threshold

現在 threshold 是 slider 手動調。可以加一個功能：

- 掃描一個範圍（例：0.3~0.9，每 0.05 一格），
- 對每個 threshold 計算 accuracy，
- 畫一張 threshold-accuracy 曲線 + 自動推薦最佳值。

這個在報告裡很好寫：

「我們比較不同門檻值對準確率的影響，最佳值約為 0.xx。」

(2) Confusion Matrix

你現在有 `results` DataFrame，裡面有：

- 實際成員

- AI 預測

可以用 pandas 做一個 confusion matrix：

```
python

cm = pd.crosstab(results["實際成員"], results["AI 預測"], rownames=["實際"], colnames=["預測"],
dropna=False)
st.dataframe(cm)
```

這樣一眼就看得出來：

- 哪兩個成員最容易被互相搞混
- 可以當作「改進資料集」的依據（例如那兩個人多準備幾張不同角度照片）

(3) 人臉特徵可視化（進階）

如果你想秀比較「研究味」的東西，可以：

- 把每個成員的 embedding 做平均後，用 t-SNE / PCA 降到 2D
- 畫散點圖，每個點一個成員名字
- 看看同一成員是不是有群聚，不同成員距離如何

這部分可以在 Notebook 做，再把圖貼回報告就很香。

3 UI / 體驗：更好玩也更易用

(1) PK 題目前後「解析區」

提交答案後除了現在那句文字外，可以再：

- 顯示 小圖對照：
 - 「正解」的幾張典型照片 vs 這張題目
- 顯示 AI 對所有成員的排序（你已經有 `rank_members`，直接列出 top 3）

這樣變成真的「教學型小遊戲」，不是只有對錯。

(2) 更明顯的狀態 / 回合進度條

你現在有文字版：

```
text

已玩：X / max_rounds
```

可以再加一個 `st.progress` 或 `st.slider` show read-only：

```
python

st.progress(quiz_state. rounds / max_rounds)
```

視覺上比較有「打關卡」的感覺。

4 Lucky Vicky 區：變成一個「文案小工具」

現在是單一 style，可以延伸：

(1) 風格模板

- 在 Lucky Vicky Tab 加一個 風格 selectbox，例如：
 - 「IG 貼文」
 - 「限動文字」
 - 「貼給好朋友的 Line 訊息」
- 根據選擇去組不同的 system prompt。

(2) 儲存「我的幸運語錄」

- 把每次生成結果 append 到一個 DataFrame
- 提供下載成 txt/markdown 或 CSV 的按鈕
- 甚至可以做個「歷史紀錄區」讓使用者再點開看。

5 效能 & 工程：讓系統更順、更穩

(1) 快取每張照片的 embedding

現在每次 PK / 測試都會重新跑 `extract_face_embedding`，如果照片多會有點慢。
可以用 `@st.cache_data`：

```
python

@st.cache_data(show_spinner=False)
def cached_embedding(image_path: str) -> Optional[np.ndarray]:
    return extract_face_embedding(model_app, image_path)
```

然後所有呼叫都改成 `cached_embedding(path)`。
這樣同一張圖不會重算，互動會順很多。

(2) 資料夾錯誤偵測

例如當某個成員沒有任何照片時：

- 在 Tab1 就用紅字提醒：哪些成員「照片數 = 0」

- PK 題庫準備時可以直接忽略沒有照片的成員，避免之後抽不到題目。

如果你要「其中一個方向」的**完整修改版程式**（例如：加難度模式 + conf matrix），跟我說你想先做哪一塊，我可以直接幫你把那一塊的 code 寫好（可以貼上取代的那種）。



好全部都改

我直接幫你剛剛講的那些功能都塞進去，包含：

- PK 題庫改用 **photos**
- 難度模式（Easy / Normal / Hard：Easy 減少選項 + 偏高信心圖片、Hard 偏低信心圖片）
- 錯題練習模式 + 歷史紀錄 & 玩家排行榜
- 進度條顯示回合進度
- 單張／全部測試後：自動掃描最佳門檻 + Confusion Matrix
- 人臉特徵 PCA 2D 可視化
- 快取人臉 embedding（加速）
- Lucky Vicky：風格範本 + 歷史語錄下載

你可以直接把下面整份程式覆蓋原本的 .py 檔，**streamlit run** 之後玩玩看：

```
python

from __future__ import annotations

import glob
import io
import os
import random
import tempfile
import zipfile
from dataclasses import dataclass, field
from datetime import datetime
from typing import Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image

try:
    import cv2
except ImportError:
    cv2 = None

try:
    from insightface.app import FaceAnalysis
except ImportError:
    FaceAnalysis = None

try:
    import aisuite as ai
```

```

except ImportError:
    ai = None

st.set_page_config(page_title="IVE PK + Lucky Vicky Hub", layout="wide")

# ===== 模型載入 & 影像工具 =====
@st.cache_resource(show_spinner=False)
def load_face_model() -> Optional[FaceAnalysis]:
    """Load InsightFace FaceAnalysis model once per session."""
    if FaceAnalysis is None:
        return None

    app = FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])
    app.prepare(ctx_id=0, det_size=(640, 640))
    return app

def ensure_folder_structure(members: List[str], base_dirs: List[str]) -> List[str]:
    created = []
    for base in base_dirs:
        for member in members:
            path = os.path.join(base, member)
            os.makedirs(path, exist_ok=True)
            created.append(path)
    return created

def iter_images(folder: str) -> List[str]:
    return [
        path
        for path in glob.glob(os.path.join(folder, "*"))
        if path.lower().endswith((".jpg", ".jpeg", ".png", ".bmp"))
    ]

def summarize_folder(folder: str, members: List[str], member_dict: Dict[str, str]) ->
pd.DataFrame:
    rows = []
    for member in members:
        path = os.path.join(folder, member)
        count = len(iter_images(path)) if os.path.isdir(path) else 0
        rows.append(
            {
                "英文名稱": member,
                "顯示名稱": member_dict.get(member, ""),
                "照片數": count,
                "路徑": os.path.abspath(path),
            }
        )
    return pd.DataFrame(rows)

def sample_member_images(
    base_dir: str,
    members: List[str],
    member_dict: Dict[str, str],
    per_member: int = 1,
    limit: int = 12,
) -> List[Dict[str, str]]:
    """Pick a few images per member so the UI can show quick previews."""
    samples: List[Dict[str, str]] = []
    for member in members:
        folder = os.path.join(base_dir, member)
        candidates = iter_images(folder)
        if not candidates:
            continue
        picks = random.sample(candidates, k=min(per_member, len(candidates)))
        for path in picks:
            samples.append(
                {
                    "path": path,

```

```

        "member_en": member,
        "member_zh": member_dict.get(member, member),
    }
)
random.shuffle(samples)
return samples[:limit]

def extract_face_embedding(app: FaceAnalysis, image_path: str) -> Optional[np.ndarray]:
    """原始版本（保留），實際使用改成快取版 get_face_embedding。"""
    if app is None or cv2 is None:
        return None

    img = cv2.imread(image_path)
    if img is None:
        return None

    faces = app.get(img)
    if not faces:
        return None

    return faces[0].embedding

@st.cache_data(show_spinner=False)
def get_face_embedding(image_path: str) -> Optional[np.ndarray]:
    """快取版的人臉特徵抽取，避免同一張圖一再計算。"""
    app = load_face_model()
    if app is None or cv2 is None:
        return None

    img = cv2.imread(image_path)
    if img is None:
        return None

    faces = app.get(img)
    if not faces:
        return None

    return faces[0].embedding

# ===== 特徵資料庫 & 評估 =====
def build_face_database(
    app: FaceAnalysis, # 參數保留但實際不用（用快取）
    photos_dir: str,
    members: List[str],
) -> Tuple[Dict[str, np.ndarray], pd.DataFrame]:
    stats = []
    database: Dict[str, np.ndarray] = {}

    for member in members:
        folder = os.path.join(photos_dir, member)
        image_files = iter_images(folder)
        embeddings = []

        for path in image_files:
            vec = get_face_embedding(path)
            if vec is not None:
                embeddings.append(vec)

        if embeddings:
            database[member] = np.mean(embeddings, axis=0)

    stats.append(
        {
            "英文名稱": member,
            "訓練照片": len(image_files),
            "成功提取": len(embeddings),
        }
    )

    return database, pd.DataFrame(stats)

```



```
def recognize_member(
    face_db: Dict[str, np.ndarray],
    features: Optional[np.ndarray],
) -> Tuple[Optional[str], float]:
    if not face_db or features is None:
        return None, 0.0

    best_match = None
    best_score = float("inf")

    for member, db_vec in face_db.items():
        denom = float(np.linalg.norm(features) * np.linalg.norm(db_vec))
        if denom == 0:
            continue
        similarity = float(np.dot(features, db_vec) / denom)
        distance = 1 - similarity
        if distance < best_score:
            best_score = distance
            best_match = member

    confidence = 1 - best_score
    return best_match, confidence

def rank_members(
    face_db: Dict[str, np.ndarray],
    features: Optional[np.ndarray],
    top_k: int = 3,
) -> List[Tuple[str, float]]:
    if not face_db or features is None:
        return []

    scores: List[Tuple[str, float]] = []
    for member, db_vec in face_db.items():
        denom = float(np.linalg.norm(features) * np.linalg.norm(db_vec))
        if denom == 0:
            continue
        similarity = float(np.dot(features, db_vec) / denom)
        scores.append((member, similarity))

    # Sort by confidence descending
    scores.sort(key=lambda item: item[1], reverse=True)
    return scores[:top_k]

def evaluate_testset(
    app: FaceAnalysis, # 參數保留但實際不用
    face_db: Dict[str, np.ndarray],
    test_dir: str,
    members: List[str],
    member_dict: Dict[str, str],
    threshold: float,
) -> pd.DataFrame:
    rows = []

    for member in members:
        folder = os.path.join(test_dir, member)
        for path in iter_images(folder):
            features = get_face_embedding(path)
            prediction, confidence = recognize_member(face_db, features)
            predicted_zh = member_dict.get(prediction or "", "無法辨識")
            actual_zh = member_dict.get(member, member)
            is_correct = bool(prediction == member and confidence >= threshold)

            rows.append(
                {
                    "圖片": os.path.basename(path),
                    "實際成員_en": member,
                    "實際成員": actual_zh,
                    "預測成員_en": prediction or "",
                    "AI 預測": predicted_zh if prediction and confidence >= threshold else "信心"
                }
            )
```

```

不足",
        "信心度": round(confidence, 3),
        "結果": "✅ 正確" if is_correct else "❌ 錯誤",
        "來源": os.path.abspath(path),
    }
)

return pd.DataFrame(rows)

def summarize_accuracy(results: pd.DataFrame) -> pd.DataFrame:
    if results.empty:
        return pd.DataFrame()

    temp = results.copy()
    temp["正確"] = temp["結果"] == "✅ 正確"
    summary = temp.groupby("實際成員").agg(
        總張數=("圖片", "count"),
        答對張數=("正確", "sum"),
        平均信心=("信心度", "mean"),
    )
    summary["準確率(%)"] = (summary["答對張數"] / summary["總張數"] * 100).round(1)
    summary["平均信心"] = summary["平均信心"].round(3)
    return summary.reset_index()

def threshold_sweep(
    results: pd.DataFrame,
    start: float,
    end: float,
    step: float,
) -> pd.DataFrame:
    """不同門檻對準確率與 coverage 的影響。"""
    if results.empty:
        return pd.DataFrame()

    rows = []
    total = len(results)
    t = start
    while t <= end + 1e-9:
        correct_mask = (
            (results["預測成員_en"] == results["實際成員_en"])
            & (results["信心度"] >= t)
        )
        conf_mask = results["信心度"] >= t

        correct = int(correct_mask.sum())
        covered = int(conf_mask.sum())
        accuracy = correct / total * 100 if total else 0.0
        coverage = covered / total * 100 if total else 0.0

        rows.append(
            {
                "門檻": round(t, 3),
                "準確率(%)": round(accuracy, 1),
                "啟用樣本比例(%)": round(coverage, 1),
            }
        )
        t += step

    return pd.DataFrame(rows)

def confusion_matrix_df(results: pd.DataFrame) -> pd.DataFrame:
    if results.empty:
        return pd.DataFrame()
    return pd.crosstab(
        results["實際成員"],
        results["AI 預測"],
    )

```

```

        rownames=["實際"],
        colnames=["AI 預測"],
        dropna=False,
    )

def pca_face_db(face_db: Dict[str, np.ndarray], member_dict: Dict[str, str]) -> pd.DataFrame:
    """用 PCA 把每位成員的平均特徵壓成 2D, 方便視覺化。"""
    if not face_db:
        return pd.DataFrame()

    names_en = list(face_db.keys())
    X = np.stack([face_db[name] for name in names_en], axis=0)
    X = X - X.mean(axis=0, keepdims=True) # 中心化

    # SVD 做 PCA
    _, _, vt = np.linalg.svd(X, full_matrices=False)
    components = X @ vt[:2].T # (n, 2)

    rows = []
    for i, name_en in enumerate(names_en):
        rows.append(
            {
                "成員英文": name_en,
                "成員": member_dict.get(name_en, name_en),
                "x": float(components[i, 0]),
                "y": float(components[i, 1]),
            }
        )
    return pd.DataFrame(rows)

def unzip_dataset(upload, target_dir: str) -> None:
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp.write(upload.read())
        tmp_path = tmp.name

    with zipfile.ZipFile(tmp_path, "r") as zip_ref:
        zip_ref.extractall(target_dir)

    os.remove(tmp_path)

# ===== PK 狀態 =====
@dataclass
class QuizState:
    pool: List[Dict[str, str]]
    current: Optional[Dict[str, str]]
    user_score: int = 0
    ai_score: int = 0
    rounds: int = 0
    last_result: str = ""
    history: List[Dict[str, str]] = field(default_factory=list)
    wrong_pool: List[Dict[str, str]] = field(default_factory=list)
    mode: str = "all" # all / wrong_only

def prepare_quiz_pool(base_dir: str, members: List[str], member_dict: Dict[str, str]) -> List[Dict[str, str]]:
    pool = []
    for member in members:
        folder = os.path.join(base_dir, member)
        for path in iter_images(folder):
            pool.append(
                {
                    "path": path,
                    "member_en": member,
                    "member_zh": member_dict.get(member, member),
                }
            )
    random.shuffle(pool)
    return pool

```

```
def pick_next_question(
    pool: List[Dict[str, str]],
    face_db: Dict[str, np.ndarray],
    difficulty: str,
) -> Optional[Dict[str, str]]:
    """依難度挑題：
    - Normal：完全隨機
    - Easy：盡量挑 AI 信心高的
    - Hard：盡量挑 AI 信心偏低的
    """
    if not pool:
        return None
    if not face_db or difficulty == "Normal":
        return random.choice(pool)

    easy_thr = 0.8
    hard_thr = 0.6
    max_tries = min(30, len(pool) * 2)

    chosen = None
    for _ in range(max_tries):
        cand = random.choice(pool)
        features = get_face_embedding(cand["path"])
        _, conf = recognize_member(face_db, features)

        if difficulty == "Easy" and conf >= easy_thr:
            chosen = cand
            break
        if difficulty == "Hard" and 0.0 < conf <= hard_thr:
            chosen = cand
            break

    if chosen is None:
        chosen = random.choice(pool)
    return chosen

# ===== LLM 呼叫 =====
def lucky_vicky_post(
    system_prompt: str,
    user_prompt: str,
    provider: str,
    model: str,
) -> str:
    if ai is None:
        raise RuntimeError("尚未安裝 aisuite, 請先執行 `pip install aisuite[all]`。")

    client = ai.Client()
    response = client.chat.completions.create(
        model=f"{provider}:{model}",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

def require_packages() -> bool:
    missing = []
    if FaceAnalysis is None:
        missing.append("insightface")
    if cv2 is None:
        missing.append("opencv-python")

    if missing:
        st.warning(
            f"缺少套件：{'', '.join(missing)}。請先執行 "
            "`pip install insightface onnxruntime opencv-python` 後再啟動 Streamlit。 "
        )
        return False
    return True
```

```
# ===== 主程式 =====
def main() -> None:
    st.title("IVE 成員 PK + Lucky Vicky Hub")
    st.caption("結合 InsightFace PK 與 AISuite Lucky Vicky 生成器的一站式 Streamlit 介面。")

    default_en = "yujin,wonyoung,gaeul,rei,liz,leeseo"
    default_zh = "兪真유진,員瑛원영,秋天가을,Rei레이,Liz리즈,李瑞이서"

    with st.sidebar:
        st.header("基本設定")
        members_en_str = st.text_input("成員英文名稱 (逗號隔開)", default_en)
        members_zh_str = st.text_input("成員顯示名稱 (逗號隔開)", default_zh)
        photos_dir = st.text_input("訓練照片資料夾", "photos")
        test_dir = st.text_input("測試照片資料夾", "test_photos")
        player_name = st.text_input("玩家名稱", "Player1")
        difficulty = st.selectbox("PK 難度", ["Easy", "Normal", "Hard"], index=1)
        threshold = st.slider("AI 信心門檻", min_value=0.3, max_value=0.95, value=0.6, step=0.05)
        max_rounds = st.number_input("PK 回合數", min_value=3, max_value=30, value=10, step=1)

        st.divider()
        st.markdown("#### Lucky Vicky API 設定")
        provider_options = {
            "mistral": {"env": "MISTRAL_API_KEY", "models": ["mistral-8b-latest", "mistral-
large-latest"]},
            "groq": {
                "env": "GROQ_API_KEY",
                "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
            },
            "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-
mini"]},
        }
        provider = st.selectbox("模型供應商", list(provider_options.keys()))
        model = st.selectbox("模型", provider_options[provider]["models"])
        HARCODED_KEYS = {
            # "openai": "你的 OPENAI KEY",
            # "groq": "你的 GROQ KEY",
            "mistral": "h97MT06BhbTrKUqDzEq4zjVenhpc32Bm",
        }

        api_key = HARCODED_KEYS.get(provider, "")
        if api_key:
            os.environ[provider_options[provider]["env"]] = api_key.strip()
            st.caption(f"已載入預設金鑰: {provider} -> {api_key}")
        else:
            st.warning("目前這個供應商沒有設定金鑰，請在程式中填入 HARCODED_KEYS 或自行設環境變數。")

    members_en = [m.strip() for m in members_en_str.split(",") if m.strip()]
    members_zh = [m.strip() for m in members_zh_str.split(",") if m.strip()]

    if len(members_en) != len(members_zh):
        st.error("英文名稱與顯示名稱數量不一致，請重新確認設定。")
        return

    member_dict = dict(zip(members_en, members_zh))

    if "face_db" not in st.session_state:
        st.session_state["face_db"] = {}
    if "db_stats" not in st.session_state:
        st.session_state["db_stats"] = pd.DataFrame()
    if "test_results" not in st.session_state:
        st.session_state["test_results"] = pd.DataFrame()
    if "quiz_state" not in st.session_state:
        st.session_state["quiz_state"] = QuizState(pool=[], current=None)
    if "lucky_history" not in st.session_state:
        st.session_state["lucky_history"] = []

    require_packages()
```

```

model_app = load_face_model()

tabs = st.tabs(
    [
        "1 資料夾管理",
        "2 人臉特徵 & 測試",
        "3 IVE PK 小遊戲",
        "4 Lucky Vicky 生成器",
    ]
)

# ----- Tab 1: dataset helper -----
with tabs[0]:
    st.subheader("資料夾管理與資料準備")
    st.markdown(
        """
        1. 建立 `photos` 與 `test_photos` 的成員子資料夾。
        2. 可直接上傳 zip (例如 Demo03v 的 photos.zip) 並解壓縮到指定資料夾。
        3. 下方會顯示當前資料夾的照片數量統計。
        """
    )

    col1, col2 = st.columns(2)
    with col1:
        if st.button("建立資料夾結構"):
            ensure_folder_structure(members_en, [photos_dir, test_dir])
            st.success("資料夾建立完成，請開始放入照片。")

    with col2:
        upload = st.file_uploader("上傳 photos/test_photos zip", type="zip")
        if upload is not None:
            unzip_dataset(upload, ".")
            st.success("解壓縮完成，請到下方查看統計。")

    st.markdown("##### 訓練照片概況")
    train_df = summarize_folder(photos_dir, members_en, member_dict)
    st.dataframe(train_df, use_container_width=True)
    lack_train = train_df[train_df["照片數"] == 0]["顯示名稱"].tolist()
    if lack_train:
        st.warning(f"訓練資料夾中以下成員目前沒有照片：{'', '.join(lack_train)}")

    st.markdown("##### 測試照片概況")
    test_df = summarize_folder(test_dir, members_en, member_dict)
    st.dataframe(test_df, use_container_width=True)
    lack_test = test_df[test_df["照片數"] == 0]["顯示名稱"].tolist()
    if lack_test:
        st.info(f"測試資料夾中以下成員目前沒有照片：{'', '.join(lack_test)}")

    st.markdown("##### 照片快速檢視")
    with st.expander("隨機抽樣，檢查資料品質"):
        sample_from = st.radio("抽樣來源", ["訓練 photos", "測試 test_photos"],
        horizontal=True)
        per_member = st.slider("每位成員抽幾張?", 0, 3, 1, step=1)
        total_limit = st.slider("最多顯示張數", 3, 18, 9, step=3)
        if st.button("抽樣預覽", type="primary"):
            base_dir = photos_dir if sample_from == "訓練 photos" else test_dir
            samples = sample_member_images(
                base_dir,
                members_en,
                member_dict,
                per_member=per_member,
                limit=total_limit,
            )
            if not samples:
                st.info("目前沒有可抽樣的照片，請先放入影像。")
            else:
                for i in range(0, len(samples), 3):
                    cols = st.columns(3)

```

```

        for col, sample in zip(cols, samples[i : i + 3]):
            caption = f"{sample['member_zh']} | {os.path.basename(sample['path'])}"
            col.image(sample["path"], caption=caption, use_container_width=True)

# ----- Tab 2: feature building + evaluation -----
with tabs[1]:
    st.subheader("InsightFace 特徵建立與測試")
    if model_app is None:
        st.warning("尚未載入 InsightFace 模型，請確認必要套件已安裝。")
    else:
        if st.button("重新計算人臉特徵資料庫"):
            with st.spinner("建立資料庫中..."):
                face_db, stats = build_face_database(model_app, photos_dir, members_en)
                st.session_state["face_db"] = face_db
                st.session_state["db_stats"] = stats
                st.success(f"完成！成功建立 {len(face_db)} 位成員的特徵。")

            if not st.session_state["face_db"]:
                st.info("請先建立人臉特徵資料庫。")
            else:
                st.markdown("##### 特徵建立結果")
                st.dataframe(st.session_state["db_stats"], use_container_width=True)

            if st.button("測試 test_photos 準確率"):
                with st.spinner("評估中..."):
                    results = evaluate_testset(
                        model_app,
                        st.session_state["face_db"],
                        test_dir,
                        members_en,
                        member_dict,
                        threshold,
                    )
                    st.session_state["test_results"] = results

                if not st.session_state["test_results"].empty:
                    results = st.session_state["test_results"]
                    st.markdown(f"##### 測試結果，共 {len(results)} 張")
                    correct = (results["結果"] == "✅ 正確").sum()
                    accuracy = correct / len(results) * 100 if len(results) else 0
                    st.metric("準確率", f"{accuracy:.1f}%")
                    st.dataframe(results, use_container_width=True)
                    st.download_button(
                        "下載逐張結果 (CSV)",
                        results.to_csv(index=False).encode("utf-8"),
                        file_name="test_results.csv",
                        mime="text/csv",
                    )

                with st.expander("不同門檻的準確率分析"):
                    start_thr = st.number_input("起始門檻", 0.0, 1.0, 0.3, 0.05)
                    end_thr = st.number_input("結束門檻", 0.0, 1.0, 0.95, 0.05)
                    step_thr = st.number_input("步進", 0.01, 0.5, 0.05, 0.01)
                    if st.button("計算最佳門檻"):
                        sweep_df = threshold_sweep(results, start_thr, end_thr, step_thr)
                        if sweep_df.empty:
                            st.info("沒有資料可以分析。")
                        else:
                            st.dataframe(sweep_df, use_container_width=True)
                            st.line_chart(
                                sweep_df.set_index("門檻")[["準確率(%)", "啟用樣本比例(%)"]]
                            )
                            best_row = sweep_df.loc[sweep_df["準確率(%)"].idxmax()]
                            st.success(
                                f"建議門檻約為 {best_row['門檻']:.2f}, "
                                f"準確率 {best_row['準確率(%)']:.1f}%, "
                                f"啟用樣本比例 {best_row['啟用樣本比例(%)']:.1f}%"
                            )

```

```

    )

    with st.expander("Confusion Matrix (混淆矩陣)"):
        cm = confusion_matrix_df(results)
        if cm.empty:
            st.info("目前沒有可用的結果。")
        else:
            st.dataframe(cm, use_container_width=True)

    summary = summarize_accuracy(results)
    if not summary.empty:
        st.markdown("##### 各成員表現")
        summary = summary.sort_values(by="準確率(%)", ascending=True)
        st.dataframe(summary, use_container_width=True)
        st.download_button(
            "下載成員摘要 (CSV)",
            summary.to_csv(index=False).encode("utf-8"),
            file_name="member_summary.csv",
            mime="text/csv",
        )

    with st.expander("人臉特徵 PCA 2D 可視化"):
        pca_df = pca_face_db(st.session_state["face_db"], member_dict)
        if pca_df.empty:
            st.caption("目前還沒有特徵資料。")
        else:
            st.scatter_chart(pca_df, x="x", y="y", color="成員")
            st.dataframe(pca_df[["成員", "x", "y"]], use_container_width=True)

    st.divider()
    st.markdown("##### 即時單張測試")
    test_image = st.file_uploader("上傳單張照片進行辨識", type=["jpg", "jpeg", "png"],
key="single_test")
    if test_image is not None:
        image_bytes = test_image.read()
        image = Image.open(io.BytesIO(image_bytes))
        st.image(image, caption="上傳圖片", width=300)

        temp_path = os.path.join(tempfile.gettempdir(),
f"streamlit_test_{test_image.name}")
        with open(temp_path, "wb") as f:
            f.write(image_bytes)

        features = get_face_embedding(temp_path)
        prediction, conf = recognize_member(st.session_state["face_db"], features)
        predicted_name = member_dict.get(prediction or "", "無法辨識")

        if prediction and conf >= threshold:
            st.success(f"AI 預測: {predicted_name} (信心度 {conf:.2f})")
        else:
            st.error("無法辨識, 請嘗試清晰的單人照片。")

# ----- Tab 3: PK mini-game -----
with tabs[2]:
    st.subheader("和 AI PK 誰比較會認 IVE 成員")
    quiz_state: QuizState = st.session_state["quiz_state"]

    col1, col2, col3 = st.columns(3)
    with col1:
        if st.button("重新載入測試題庫"):
            quiz_state.pool = prepare_quiz_pool(photos_dir, members_en, member_dict)
            quiz_state.current = None
            quiz_state.user_score = 0
            quiz_state.ai_score = 0
            quiz_state.rounds = 0
            quiz_state.last_result = ""
            quiz_state.history.clear()
            quiz_state.wrong_pool.clear()
            quiz_state.mode = "all"
            st.toast(f"載入 {len(quiz_state.pool)} 題完成! (來源: {photos_dir})")

```



```

with col2:
    if st.button("下一題", disabled=not quiz_state.pool):
        quiz_state.current = pick_next_question(
            quiz_state.pool,
            st.session_state["face_db"],
            difficulty,
        )
        st.session_state["user_choice"] = ""

with col3:
    if st.button("重置分數"):
        quiz_state.user_score = 0
        quiz_state.ai_score = 0
        quiz_state.rounds = 0
        quiz_state.last_result = ""

col4, col5 = st.columns(2)
with col4:
    if st.button("只出我答錯的題目", disabled=not quiz_state.wrong_pool):
        quiz_state.pool = list(quiz_state.wrong_pool)
        quiz_state.mode = "wrong_only"
        quiz_state.current = None
        st.toast(f"已切換到錯題練習模式，共 {len(quiz_state.pool)} 題。")
with col5:
    if st.button("改回全部題庫", disabled=False):
        quiz_state.pool = prepare_quiz_pool(photos_dir, members_en, member_dict)
        quiz_state.mode = "all"
        quiz_state.current = None
        st.toast(f"已改回全部題庫，總題數 {len(quiz_state.pool)}。")

if not st.session_state["face_db"]:
    st.info("請先在『人臉特徵 & 測試』分頁建立人臉特徵資料庫，AI 才能參與 PK。")
elif not quiz_state.pool:
    st.info("請先按「重新載入測試題庫」。")
else:
    with st.expander("先複習一下各成員長相"):
        study_samples = sample_member_images(photos_dir, members_en, member_dict,
per_member=1, limit=6)
        if not study_samples:
            st.caption("資料夾裡還沒有照片可供複習。")
        else:
            for i in range(0, len(study_samples), 3):
                cols = st.columns(3)
                for col, sample in zip(cols, study_samples[i : i + 3]):
                    col.image(sample["path"], caption=sample["member_zh"],
use_container_width=True)

all_names = list(member_dict.values())
if not all_names:
    st.warning("沒有可選的成員名稱，請先在側欄設定成員名單。")
elif quiz_state.current:
    st.image(quiz_state.current["path"], caption="請猜猜這是哪位成員？", width=320)
    with st.expander("需要提示嗎？"):
        if st.button("AI 給我兩個可能的人選"):
            features = get_face_embedding(quiz_state.current["path"])
            top_candidates = rank_members(st.session_state["face_db"], features,
top_k=2)

            if not top_candidates:
                st.info("目前無法給提示，可能影像品質不佳。")
            else:
                for name, conf in top_candidates:
                    st.write(f"{member_dict.get(name, name)} (信心 {conf:.2f})")

    correct_answer = quiz_state.current["member_zh"]
    if difficulty == "Easy":
        others = [name for name in all_names if name != correct_answer]
        random.shuffle(others)
        options = [correct_answer] + others[:2]
        random.shuffle(options)

```

```

else:
    options = all_names

# 清理舊的 radio 選項
if (
    "user_choice" in st.session_state
    and st.session_state["user_choice"] not in options
):
    del st.session_state["user_choice"]

with st.form("pk_form"):
    user_choice = st.radio(
        f"你的答案 (難度: {difficulty}) ",
        options=options,
        key="user_choice",
        horizontal=True,
    )
    submitted = st.form_submit_button("提交答案")

if submitted and user_choice:
    features = get_face_embedding(quiz_state.current["path"])
    prediction, conf = recognize_member(st.session_state["face_db"], features)
    ai_choice = member_dict.get(prediction or "", "無法辨識")
    correct_answer = quiz_state.current["member_zh"]

    user_correct = user_choice == correct_answer
    ai_correct = prediction == quiz_state.current["member_en"] and conf >=
threshold

    quiz_state.rounds += 1
    quiz_state.user_score += int(user_correct)
    quiz_state.ai_score += int(ai_correct)

    quiz_state.last_result = (
        f"第 {quiz_state.rounds} 回合: 正解 {correct_answer}\n"
        f"你回答 {user_choice} {'✅' if user_correct else '❌'};"
        f"AI 回答 {ai_choice} (信心 {conf:.2f}) {'✅' if ai_correct else '❌'}"
    )

# 紀錄歷史
quiz_state.history.append(
    {
        "時間": datetime.now().strftime("%H:%M:%S"),
        "玩家": player_name,
        "圖片": os.path.basename(quiz_state.current["path"]),
        "正解": correct_answer,
        "玩家答案": user_choice,
        "AI 答案": ai_choice,
        "AI 信心": round(conf, 3),
        "玩家是否答對": user_correct,
        "AI 是否答對": ai_correct,
    }
)

# 收集錯題
if not user_correct:
    if all(item["path"] != quiz_state.current["path"] for item in
quiz_state.wrong_pool):
        quiz_state.wrong_pool.append(quiz_state.current)

if quiz_state.rounds >= max_rounds:
    if quiz_state.user_score > quiz_state.ai_score:
        quiz_state.last_result += "\n🏆 你戰勝 AI !"
    elif quiz_state.user_score < quiz_state.ai_score:
        quiz_state.last_result += "\n🤖 AI 更勝一籌, 下次加油 !"
    else:
        quiz_state.last_result += "\n🤝 平手, 勢均力敵 !"

# 分數 & 進度

```

```

progress = quiz_state. rounds / max_rounds if max_rounds else 0.0
st.progress(min(progress, 1.0))

st.markdown(
    f"""
        **目前比分 (玩家: {player_name}) **
        🧑 玩家: {quiz_state.user_score} 分
        🤖 AI: {quiz_state.ai_score} 分
        已玩: {quiz_state. rounds}/{max_rounds} 回合
        題庫模式: {"錯題練習" if quiz_state.mode == "wrong_only" else "全部題目"}
    """
)

if quiz_state.last_result:
    st.info(quiz_state.last_result)

if quiz_state.history:
    with st.expander("歷史紀錄 & 玩家表現"):
        hist_df = pd.DataFrame(quiz_state.history)
        st.dataframe(hist_df, use_container_width=True)
        summary = (
            hist_df.groupby("玩家")
            .agg(
                作答題數=("圖片", "count"),
                玩家答對=("玩家是否答對", "sum"),
                AI答對=("AI 是否答對", "sum"),
            )
            .assign(
                玩家正確率=lambda d: (d["玩家答對"] / d["作答題數"] * 100).round(1),
                AI正確率=lambda d: (d["AI答對"] / d["作答題數"] * 100).round(1),
            )
        )
        st.markdown("##### 玩家排行榜 (依玩家正確率排序)")
        st.dataframe(
            summary.sort_values("玩家正確率", ascending=False),
            use_container_width=True,
        )

# ----- Tab 4: Lucky Vicky generator -----
with tabs[3]:
    st.subheader("Lucky Vicky 員瑛式思考生成器 ")

    style_templates = {
        "IG 貼文": (
            "請用台灣習慣的中文來寫這段 IG 貼文:\n"
            "請用員瑛式思考, 什麼都正向思維任何使用者的事情, "
            "用第一人稱社群媒體口吻說一次, 說明為什麼這是一件超幸運的事, "
            "語氣可以可愛、有點碎念, 但要溫暖, "
            "最後以「完全是 Lucky Vicky 呀!」結尾, 可以加入 emoji。"
        ),
        "限時動態": (
            "請用很口語、短句的方式, 像 IG 限時動態那樣, "
            "把使用者的事情當成一個超好笑又超幸運的小插曲, "
            "句子可以分行, 適合貼在限時動態文字, "
            "最後加上一句「完全是 Lucky Vicky 呀!」與幾個適合的 emoji。"
        ),
        "Line 給朋友": (
            "請用像在 Line 上跟好朋友聊天的口吻, "
            "一邊吐槽、一邊安慰、一邊把事情轉成好運, "
            "適合直接複製貼給朋友看, "
            "最後一句是「完全是 Lucky Vicky 呀!」並加上一兩個 emoji。"
        ),
    }

    style_choice = st.selectbox("風格範本", list(style_templates.keys()))
    system_prompt = st.text_area(
        "System Prompt",

```

```

        value=style_templates[style_choice],
        height=180,
    )
    user_prompt = st.text_area("請輸入想轉念的事件", height=120, placeholder="例如：今天咖啡灑到電腦上了...")

    if st.button("Lucky Vicky 魔法 ✨"):
        if not api_key:
            st.error("請先在左側設定 API 金鑰。")
        elif not user_prompt.strip():
            st.error("請輸入事件描述。")
        else:
            try:
                with st.spinner("生成中..."):
                    result = lucky_vicky_post(system_prompt, user_prompt, provider, model)
                st.success("生成完成！")
                st.write(result)

                st.session_state["lucky_history"].append(
                    {
                        "時間": datetime.now().strftime("%H:%M:%S"),
                        "provider": provider,
                        "model": model,
                        "事件": user_prompt.strip(),
                        "回覆": result,
                    }
                )
            except Exception as exc: # noqa: BLE001
                st.error(f"呼叫 LLM 時發生錯誤：{exc}")

    if st.session_state["lucky_history"]:
        with st.expander("查看歷史 Lucky Vicky 語錄"):
            hist_df = pd.DataFrame(st.session_state["lucky_history"])
            st.dataframe(hist_df[["時間", "provider", "model", "事件"]],
                use_container_width=True)
            st.download_button(
                "下載完整語錄 (CSV)",
                hist_df.to_csv(index=False).encode("utf-8"),
                file_name="lucky_vicky_history.csv",
                mime="text/csv",
            )

if __name__ == "__main__":
    main()

```

如果你跑起來哪一段有 error（像是某些欄位名稱打錯、型別不合、或 Streamlit 版號不支援某功能），把錯誤訊息貼上來，我幫你一起 debug。



能幫我把這些問答紀錄輸出成md嗎

下面是這段關於 IVE PK + Lucky Vicky 專案的問答紀錄，已整理成一份 Markdown，可直接複製成 `qa_ive_pk_lucky_vicky.md` 使用。

markdown

IVE PK + Lucky Vicky Hub 問答紀錄

> 時間：2025-11

> 內容：Cursor / API 金鑰設定、Lucky Vicky API 串接、IVE PK 小遊戲題庫與功能延伸

Q1. (Cursor 畫面) 如何放 OpenAI API ?

問：

在 Cursor 看到要填 OpenAI API Key, 不知道怎麼設。

答：

1. 到瀏覽器開啟 OpenAI 平台：
<<https://platform.openai.com>>
登入你的 OpenAI 帳號。
2. 在後台找到 **API keys** 頁面，按 **Create new secret key** 產生新的 key。
系統會顯示一串 ``sk-...``，只會出現一次，要立刻複製。
3. 回到 Cursor：
 - 打開右上角齒輪 → **Settings**
 - 找到 **OpenAI API Key / API Keys** (視版本不同)
 - 將剛剛複製的 ``sk-...`` 貼到欄位，按 **Verify / Save**

> 注意：

- > - API key 視同「錢包」，不要截圖、不要上傳 GitHub。
- > - ChatGPT Plus 的額度 ≠ API 額度，API 是另外計費。

Q2. 我的 Cursor 設定畫面跟教學不一樣，找不到 Features / YOLO Auto-run

問：

教學截圖有 ``Cursor Settings > Features > Chat > Enable auto-run mode (YOLO)``，但我新版 Cursor 裡找不到這個選項。

答：

新版 Cursor UI 已改版，「Features」這個標籤拿掉了，YOLO / Auto-run 搬到 **Agents** 底下：

1. 打開 Cursor 設定：
 - 右上角齒輪 → **Cursor Settings**
2. 左側點 **Agents** (不是 Indexing & Docs)。
3. 在 Agents 頁面往下捲，找類似：
 - ``Enable auto-run mode``
 - ``Command allowlist``
 - ``Command denylist``
4. 開啟 Auto-run 開關，並在 allowlist 中只放安全指令，如：
 - ``python``
 - ``npm``
 - ``pytest``
 - ``npm run dev``
 危險指令 (例如 ``rm``, ``git reset --hard``) 可以放到 denylist。
5. Chat 面板也要設為 Agent 模式，自動跑任務才會生效。

若完全看不到 Auto-run：

- 建議到 Cursor 官網重新下載最新版覆蓋安裝，再回來看 Agents 設定。

Q3. Streamlit Lucky Vicky API 設定：金鑰要寫在哪裡？

****程式片段：****

```
```python
st.markdown("#### Lucky Vicky API 設定")
provider_options = {
 "groq": {
 "env": "GROQ_API_KEY",
 "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
 },
 "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
 "mistral": {"env": "MISTRAL_API_KEY", "models": ["ministral-8b-latest", "mistral-large-latest"]},
}
provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])
api_key = st.text_input(f"{provider_options[provider]['env']} (不會儲存)", type="password")
if api_key:
 os.environ[provider_options[provider]['env']] = api_key.strip()
```

問：

我有一組 API key（此處已省略），要寫到哪裡？

答：

這段程式的設計是：**不在程式寫死金鑰，而是讓你在網頁 UI 輸入。**

使用方式：

1. **streamlit run xxx.py** 跑起來後，在側欄找到：

- 「模型供應商」：選 **openai / groq / mistral**
- 下面會出現 **OPENAI\_API\_KEY**（不會儲存）或 **MISTRAL\_API\_KEY**（不會儲存）的輸入框。

2. 直接在這個輸入框輸入（貼上）你的 API key → **api\_key** 變數就會拿到這個值，然後：

```
python

os.environ[provider_options[provider]["env"]] = api_key.strip()
```

會把 key 設到對應的環境變數（例如 **OPENAI\_API\_KEY**）。

✅ 正常用法：

不用改程式，把 key 貼在 **Streamlit UI** 的輸入框。

## Q4. 如果真的要「寫死」在程式裡，要怎麼寫？

問：

我想直接在程式裡寫死 key（只在本機測試用），要寫在哪裡？

答：

可以用一個 **HARDCODED\_KEYS** 字典，依照 provider 拿對應 key，例如：

python

```
st.markdown("#### Lucky Vicky API 設定")
provider_options = {
 "mistral": {"env": "MISTRAL_API_KEY", "models": ["ministral-8b-latest", "mistral-large-latest"]},
 "groq": {
 "env": "GROQ_API_KEY",
 "models": ["openai/gpt-oss-120b", "llama-3.3-70b-versatile", "gemma2-9b-it"],
 },
 "openai": {"env": "OPENAI_API_KEY", "models": ["gpt-4o", "gpt-4o-mini", "gpt-4.1-mini"]},
}
provider = st.selectbox("模型供應商", list(provider_options.keys()))
model = st.selectbox("模型", provider_options[provider]["models"])

HARDCODED_KEYS = {
 # "openai": "你的 OPENAI_KEY",
 # "groq": "你的 GROQ_KEY",
 "mistral": "你的 MISTRAL_KEY",
}

api_key = HARDCODED_KEYS.get(provider, "")

if api_key:
 os.environ[provider_options[provider]["env"]] = api_key.strip()
 st.caption(f"已載入預設金鑰：{provider}")
else:
 st.warning("目前這個供應商沒有設定金鑰，請在 HARDCODED_KEYS 中填入。")
```

 強烈建議：

- 這種「寫死」方式只用於本機測試。
- 不要把含有金鑰的檔案上傳 GitHub / 分享給別人。

## Q5. IVE PK 小遊戲題庫是用哪個資料夾？

程式片段：

python

```
test_dir = st.text_input("測試照片資料夾", "test_photos")

if st.button("重新載入測試題庫"):
 quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
```

prepare\_quiz\_pool：

python

```
def prepare_quiz_pool(test_dir: str, members: List[str], member_dict: Dict[str, str]) -> List[Dict[str, str]]:
 pool = []
 for member in members:
 folder = os.path.join(test_dir, member)
 for path in iter_images(folder):
 pool.append(
 {
 "path": path,
 "member_en": member,
 "member_zh": member_dict.get(member, member),
 }
)
```

```
 }
 random.shuffle(pool)
 return pool
```

問：

IVE PK 小遊戲的題庫，是讀哪個資料夾？

答：

題庫路徑是：

{測試照片資料夾}/{成員英文名稱}/裡面的所有圖片

預設情況（側欄 測試照片資料夾 = test\_photos）就是：

- ./test\_photos/yujin/
- ./test\_photos/wonyoung/
- ./test\_photos/gaeul/
- ./test\_photos/rei/
- ./test\_photos/liz/
- ./test\_photos/leeseo/

這些子資料夾裡的 .jpg / .png / .jpeg / .bmp 影像會被放進 PK 題庫。

## Q6. 如果我想改成 photo 資料夾要怎麼改？

問：

我想把「IVE PK 小遊戲的題庫」改成用 photo 資料夾，而不是 test\_photos。

答（簡易版）：

作法一：只改 UI 的預設值（最簡單）

把這行：

```
python

test_dir = st.text_input("測試照片資料夾", "test_photos")
```

改成：

```
python

test_dir = st.text_input("測試照片資料夾", "photo")
```

然後在專案資料夾建立：

- photo/yujin/
- photo/wonyoung/



- ...

按「重新載入測試題庫」後，題庫就會讀 `photo/...` 了。

問：

但我是想「專門讓 IVE PK 小遊戲用 `photos` 當題庫」，測試準確率還是用 `test_photos`。

答（你最後採用的設計）：

在 **PK 小遊戲** 那一段，原本是：

```
python

if st.button("重新載入測試題庫"):
 quiz_state.pool = prepare_quiz_pool(test_dir, members_en, member_dict)
```

改成：

```
python

if st.button("重新載入測試題庫"):
 quiz_state.pool = prepare_quiz_pool(photos_dir, members_en, member_dict)
```

- 這樣：
  - Tab 2 「測試準確率」仍然用 `test_dir` (`test_photos`)
  - Tab 3 「IVE PK 小遊戲」的題庫改成讀 `photos_dir` (`photos`)

## Q7. 這個 IVE PK + Lucky Vicky Hub 還有延伸空間嗎？

問：

我已經有：

- 資料夾管理
- 特徵建立
- 測試準確率
- IVE PK 小遊戲
- Lucky Vicky 生成器

還有什麼可以再加？

答：（整理幾個方向）

### 1. IVE PK 小遊戲 → 做成「完整遊戲」

- 難度模式（Easy / Normal / Hard）：
  - Easy：選項只給 3 個（正解 + 2 個干擾），題目偏 AI 信心高的照片。
  - Normal：維持現在 6 選 1。

- Hard：題目從 AI 信心偏低的照片抽題。
- 錯題練習模式：
  - 記錄使用者答錯的題目，切換成「只出錯題」模式，練弱點。
- 歷史紀錄 & 玩家排行榜：
  - 記錄每一題：玩家答案、AI 答案、信心度、正確與否。
  - 依照玩家名稱統計正確率，做排行榜。
- 進度條：
  - 用 `st.progress(rounds / max_rounds)` 顯示目前玩到第幾回合。

## 2. 模型 / 分析 → 可以寫進作業報告的東西

- 門檻掃描 (Threshold Sweep)：
  - 掃描 0.3 ~ 0.9 每 0.05，計算每個門檻的準確率與 coverage。
  - 繪製 threshold-accuracy 曲線，找出最佳門檻。
- Confusion Matrix：
  - 用 `pd.crosstab(實際成員, AI 預測)` 算混淆矩陣，看哪兩個成員最常被搞混。
- PCA 可視化：
  - 把每位成員的平均 embedding 做 PCA 降到 2D。
  - 畫散點圖（每個點代表一位成員），觀察 cluster / 距離。

## 3. Lucky Vicky 區 → 變成「文案小工具」

- 風格模板：
  - IG 貼文版、限動版、Line 傳訊版，不同 system prompt。
- 歷史語錄 / 下載：
  - 每次生成都記錄，最後匯出成 CSV 或 Markdown，變成自己的「幸運語錄本」。

## 4. 效能 / 工程

- 快取 embedding：
  - 用 `@st.cache_data` 對每張照片的 embedding 做快取，遊戲過程就不會一直重算。
- 資料夾檢查：
  - 在 Tab1 直接提醒哪個成員沒有照片，避免遊戲抽到空題目。

## Q8. 全部功能整合版程式

問：  
「好，全部都改。」

能直接給一份整合所有上述功能的完整程式嗎？

答：

已提供一份完整的 `streamlit` 程式碼，包括：

- 快取版 `get_face_embedding`
- 門檻掃描 `threshold_sweep`
- Confusion Matrix 輸出
- PCA 2D 可視化
- IVE PK 小遊戲：
  - 難度模式（Easy / Normal / Hard）
  - 錯題練習模式
  - 歷史紀錄 & 玩家排行榜
  - 回合進度條
- Lucky Vicky：
  - 風格範本（IG / 限動 / Line）
  - 語錄歷史 & CSV 下載
- HARDCODED\_KEYS 放在側欄設定區

你只需要把那段完整程式貼到 `.py` 檔，執行：

`streamlit run 你的檔名.py`

即可使用。

## Q9. 把這些問答整理成 Markdown

問：

能幫我把這些問答紀錄輸出成 `.md` 嗎？

答：

本檔案即為整理後的 Markdown 版問答紀錄，可直接儲存為 `qa_ive_pk_lucky_vicky.md`，也可以貼到 GitHub Repo 當作專案的說明或開發筆記。

```
makefile
```

```
:::contentReference[oaicite:0]{index=0}
```